



152118640- ENGINEERING RESEARCH ON CYBER SECURITY

1. ARA SINAV RAPORU

Proje Başlığı

“SynGuard-SDN: Real-Time SYN Flood Detection System for Software-Defined Networks”

Projeyi Hazırlayanlar:

Umut ÖZTÜRK 152120211052
Abdullah Taha AYDIN 152120211055
Eren EROĞLU 152120211105
Atakan BERBER 152120211057

Danışman:

Dr. Öğr. Üyesi İLKER ÖZÇELİK

A. PLANLAMA.....	3
A.1 Özet (Abstract) ve Anahtar Kelimeler (Keywords)	4
A.2 Bilgi Gereksinim Belirleme, Problemin Tanımlanması.....	5
A.2.1 Amaç	5
A.2.2 Konu Ve Kapsam	6
A.2.3 Literatür Özeti.....	6
A.3 Beklenen Fayda.....	7
A.3.1 Özgün Değer	7
A.3.2 Yaygın Etki/ Katma Değer	8
A.4 Yöntem.....	8
A.5 Araştırma Yöntemi.....	8
A.6 Çalışma Takvimi	9
A.6.1 İş Zaman Çizelgesi.....	9
A.6.2 Kim Ne İş Yapacak.....	9
B. ANALİZ	10
B.1 Sistem Gereksinimlerini Ortaya Çıkarma Yöntem ve Teknikleri	10
B.1.1 Yüz Yüze Görüşme	10
B.1.2 Yazılı Basılı Belge İncelemesi.....	10
B.1.3 Anket.....	11
B.1.4 Gözlem	15
B.1.5 Veri Akış Şemaları	15
B.1.5.1 Kavramsal Veri Akış Şeması	16
B.1.5.2 Mantıksal Veri Akış Şeması	16
B.1.5.3 Fiziksel Veri Akış Şeması	17
B.1.6. Olay Tabloları, Durum Formları, İşlevsel Analiz Raporu, İş Akış Şeması	18
B.1.6.1 Olay Tabloları	18
B.1.6.2 Durum Formları	20
B.1.6.3 İşlevsel Analiz Raporu	22
B.1.6.3 İş Akış Şemaları	23
B.2.1 İşlevsel Gereksinimler	24
B.2.1.1. Veri Toplama ve Ön İşleme İşlevleri	24
B.2.1.2. Saldırı Tespit ve Analiz İşlevleri.....	24
B.2.1.3. Kullanıcı Etkileşimi ve Görselleştirme İşlevleri	25
B.2.1.4. Veri Yönetimi ve Kayıt Tutma İşlevleri	25
B.2.2. Sistem ve Kullanıcı Ara Yüzleri ile İlgili Gereksinimler	25
B.2.2.1. Kullanıcı Ara Yüzü (UI) Gereksinimleri	25
B.2.2.2. Sistem Ara Yüzleri ve Entegrasyon Gereksinimleri	25
B.2.3. Veriye İlgili Gereksinimler	26
B.2.3.1. Veri Kaynağı ve İçerik Gereksinimleri	26
B2.3.2. Veri Saklama ve Yapılandırma Gereksinimleri	26
B.2.4. Kullanıcılar ve İnsan Faktörü Gereksinimleri, Güvenlik Gereksinimleri.....	26
B.2.4.1. Kullanıcı ve İnsan Faktörü Gereksinimleri	26
B.2.4.2. Güvenlik Gereksinimleri.....	26
B.2.5. Teknik ve Kaynak Gereksinimleri, Fiziksel Gereksinimler	26
B.2.5.1. Yazılım Gereksinimleri ve Geliştirme Ortamı	26
B.2.5.2. Teknik ve Kaynak Gereksinimleri	27
B.2.5.3. Fiziksel ve Altyapı Gereksinimleri	27
C. TASARIM	27

C.1. Sistem Tasarımı.....	27
C.1.1. Genel Mimari Yapı (Split Architecture).....	27
C.1.2. Fiziksel Veri Akışı ve Katmanlar.....	28
C.1.3. Modüler Bileşen Tasarımı	28
C.2.1. Tasarım Prensipleri ve Görsel Dil	29
C.2.2. Ekran Tasarımları ve Bileşen Analizi	29
C.2.2.1. Kullanıcı Giriş Yapma.....	29
C.2.2.1. Kullanıcı Kaydı Yapma	30
C.2.2.2. Genel Bakış ve Canlı İzleme (Overview).....	30
C.2.2.3. Ağ Topolojisi ve Tehdit Haritası (Topology Map).....	31
C.2.2.4. Olay Arşivi ve Kayıt Yönetimi (Incident Logs)	31
C.2.2.5. Teknik İzleme ve Terminal (Live Terminal)	32
C.3. Test Tasarımı.....	32
C.3.1. Gereksinim Analizlerinden Teste Yönelik Hedeflerin Detaylandırılması	33
C.3.2. Fonksiyonel Test Tasarımı.....	33
C.3.3. Performans Test Tasarımı	33
C.4. Yazılım Tasarımı	33
C.4.1. Gereksinime Bağlı Tasarım Kalıpları Seçimi	34
C.4.2. UML Kullanarak Tasarım Diyagramları Oluşturma	34
C.4.2.1. Kullanım Senaryosu (Use Case) Diyagramı	34
C.4.2.2. Bileşen (Component) Diyagramı.....	35
C.4.2.3. Sıralama ve Akış (Sequence) Diyagramı.....	36
C.5. Veri Tabanı Tasarımı	37
C.5.1. Veri Tabanı İsterleri ve Tablo Dökümü	37
C.5.1.1. Users Tablosu (Kullanıcı Yönetimi)	37
C.5.1.2. TrafficLogs Tablosu (Ağ Akış Arşivi)	37
C.5.1.3. Incidents Tablosu (Saldırı Kayıtları).....	37
C.5.2. E/R (Varlık-İlişki) Yapısı.....	37
C.5.3. İş Mantığı, Kısıt (Constraint) ve Tetikleyici (Trigger) Tasarımları	38
D. REFERANSLAR	38

A. PLANLAMA

A.1 Özet (Abstract) ve Anahtar Kelimeler (Keywords)

ÖZET

Yazılım Tanımlı Ağlar (SDN), merkezi kontrol yapısı ve programlanabilir mimarisi sayesinde modern ağ yönetiminde önemli avantajlar sunmaktadır. Ancak bu yapı, özellikle SYN flood gibi hizmet engelleme temelli saldırılar karşısında ağ sürekliliğini tehdit eden kritik güvenlik riskleri de barındırmaktadır. Bu projede, SDN ortamlarında aktif akış verileri üzerinden SYN Flood saldırılarını gerçek zamanlı olarak tespit edebilen makine öğrenmesi tabanlı bir Saldırı Tespit Sistemi (IDS) tasarlanması amaçlanmaktadır.

Geliştireceğimiz bu sistemde, SDN denetleyicisinden elde edilen akış istatistikleri belirli zaman aralıklarında toplanacak; denetleyici üzerinden doğrudan elde edilebilen veriler analiz edilecektir. Bu öznitelikler, normal ve saldırı trafiğini sınıflandırmak üzere geliştirilecek model tarafından analiz edilecektir. Literatürdeki birçok çalışmanın veri seti tabanlı ve çevrimdışı analiz odaklı olması nedeniyle, gerçek ağ akışı üzerinde çalışan ve kullanıcı etkileşimini destekleyen sistemler sınırlı kalmaktadır. Bu proje, söz konusu eksikliği gidermek amacıyla gerçek zamanlı trafik analizi, alarm üretimi, geçmiş kayıt görüntüleme ve kullanıcı giriş ekranı içeren bir arayüz yapısını aynı sistem içerisinde birleştirmektedir.

Genel olarak ilgili çalışmalar değerlendirildiğinde üç temel eğilim öne çıkmaktadır. Birincisi, SDN ortamında saldırı tespiti için akış tabanlı ya da port tabanlı istatistiklerin yoğun biçimde kullanılmasıdır. İkincisi, eşik değeri ve kural tabanlı yöntemlerden makine öğrenmesi ve derin öğrenme yöntemlerine doğru belirgin bir geçiş bulunmasıdır. Üçüncüsü ise doğruluk oranı yüksek yöntemlerin her zaman gerçek zamanlı uygulanabilirlik açısından en uygun çözüm olmamasıdır. Özellikle karmaşık derin öğrenme ya da grafik tabanlı yaklaşımlar akademik olarak güçlü görünse de, canlı akış üzerinde düşük gecikme ile çalışan, kullanıcı arayüzü ile desteklenen ve operasyonel olarak izlenebilir bir IDS sistemine dönüştürülmeleri daha zor olabilmektedir.

Geliştireceğimiz sistemin doğruluk, precision, recall, F1-score ve gecikme süresi gibi performans ölçütleriyle değerlendirilmesi planlanmaktadır. Projenin temel katkısı, SDN tabanlı ağlarda SYN flood saldırılarının erken tespitine yönelik, gerçek zamanlı, izlenebilir ve kullanıcı odaklı bir IDS mimarisi önermesi olacaktır.

Anahtar Kelimeler: SDN, SYN Flood, IDS, Gerçek Zamanlı Saldırı Tespiti, Makine Öğrenmesi, Derin Öğrenme, Akış Tabanlı Analiz

ABSTRACT

Software-Defined Networks (SDN) offer significant advantages in modern network management through their centralized control structure and programmable architecture. However, this architecture also introduces critical security risks that may threaten network availability, particularly against denial-of-service attacks such as SYN flood. In this project, we aim to design a machine learning-based Intrusion Detection System (IDS) capable of detecting SYN flood attacks in real time by using active flow data collected from SDN environments.

In the proposed system, flow statistics obtained from the SDN controller will be collected at regular intervals, and the data that can be directly acquired through the controller will be analyzed. These features will then be evaluated by a model developed to classify normal and attack traffic. Since many studies in the literature focus primarily on dataset-based and offline analysis, systems that operate on real network traffic and support user interaction remain limited. To address this gap, the proposed project combines real-time traffic analysis, alarm generation, historical record visualization, and a user login interface within a single integrated system.

When the related studies are considered in general, three main trends can be observed. First, flow-based or port-based statistics are widely used for attack detection in SDN environments. Second, there is a clear shift from threshold-based and rule-based methods toward machine learning and deep learning approaches. Third,

methods with high accuracy are not always the most suitable solutions for real-time applicability. In particular, although complex deep learning or graph-based approaches appear strong from an academic perspective, they are more difficult to transform into operationally traceable IDS systems that run with low latency on live traffic and are supported by a user interface.

The developed system is planned to be evaluated using performance metrics such as accuracy, precision, recall, F1-score, and latency. The main contribution of the project will be the proposal of a real-time, observable, and user-oriented IDS architecture for the early detection of SYN flood attacks in SDN-based networks.

Keywords: SDN, SYN Flood, IDS, Real-Time Attack Detection, Machine Learning, Deep Learning, Flow-Based Analysis

A.2 Bilgi Gereksinim Belirleme, Problemin Tanımlanması

Günümüzde ağ altyapılarının, hızla gelişen teknoloji ve kümülatif şekilde artan ağ trafiği gibi etmenler dolayısıyla kusursuz bir şekilde çalışması beklenmektedir. Ağ altyapısı için kullanılan geleneksel ağ mimarilerinde her yönlendirici ve anahtar kendi kontrol düzlemi kapsamında çalışmaktadır. Bu mimariye sahip bir ortam, dağınık bir yapıya sahiptir ve merkezcil bir yapı oluşturmamaktadır. Bunun sonucunda olası saldırı senaryolarında, saldırıyı önlemek için vaktinde alınması gereken aksiyonların uygulanması kısmında zorluk çekilebilir. Yazılım Tanımlı Ağ (SDN) mimarisi, kontrol düzlemini veri düzleminden ayırarak ağı merkezi bir denetleyici üzerinden yönetilebilir hale getirir ve bu mimari, güçlü bir çözüm yöntemi sunar. Ancak kontrolün merkezileşmesi, güvenlik açısından yeni ve kritik bir zafiyet alanı yaratır. Özellikle SYN Flood gibi DDoS türündeki saldırılarda, kontrolcünün denetleyiciler tarafından meşgul edilmesi, ağ performansında düşüş ve gecikmesi, hizmet kesintileri gibi sorunlara yol açabilmektedir.

Bu yüzden SDN ortamlarında bu tür saldırılarının erken ve doğru bir şekilde tespit edilmesi son derece önemlidir. Ancak bu tespiti gerçekleştirmek için öncelikle SDN denetleyicisinden hangi verilerin doğrudan elde edilebileceği belirlenmesi gerekmektedir. Paket sayısı, bayt sayısı, akış süresi, akış sayısı, kaynak ve hedef IP adresleri, kaynak ve hedef port bilgileri ile protokol türü gibi akış istatistikleri, saldırı tespit sürecinde kritik veriler arasında yer almaktadır. Kısa süre içinde artan bağlantı talepleri, aynı hedefe yönelen yoğun trafik ve akış tablosunda meydana gelen olağandışı artışlar, bu saldırının tespitinde dikkate alınması gereken önemli göstergeler arasında yer almaktadır.

Mevcut yaklaşımlar incelendiğinde, yalnızca sabit eşik değerlerine dayanan geleneksel IDS (Intrusion Detection System) sistemlerinin yoğun fakat meşru trafik durumlarında yanlış alarm üretme olasılığının yüksek olduğu gözlemlenmiştir. Bununla birlikte, karmaşık ve ağır derin öğrenme modelleri yüksek doğruluk sunsa bile, canlı ağ ortamında karar verme süresini uzatarak gerçek zamanlı kullanımda dezavantaj oluşturabilir.

Bu problemlerin çözüme kavuşturulması için mevcut sistemler ve yaklaşımlar incelenmiştir. Anketler aracılığıyla kullanıcı geri bildirimleri elde edilmiş, incelenmiştir. Mevcut sorunlar, bu incelemeler doğrultusunda detaylandırılmıştır. Tüm bunların sonucu olarak ise, sadece SYN Flood saldırısını tespit etmek değil; bunu gerçek zamanlı, denetleyiciyi ek yük altında bırakmayacak ve denetleyiciden elde edilen verileri etkili bir şekilde işleyen kullanıcı dostu bir arayüz sunan bir IDS sisteminin geliştirilmesi gerekliliğine karar verilmiştir.

A.2.1 Amaç

Bu projenin amacı, Yazılım Tanımlı Ağ (SDN) ortamlarında TCP SYN Flood saldırılarını tespit edebilen gerçek zamanlı bir saldırı tespit sistemi geliştirmektir. Önerilen sistem, SDN denetleyicisinden elde edilen

akış verilerini kullanarak normal ağ trafiği ile saldırı trafiğini ayırt etmeyi ve saldırı durumlarını kullanıcıya anlaşılır bir şekilde sunmayı hedeflemektedir.

Geliştirilecek yapı, yalnızca saldırı tespiti yapan bir modelden oluşmayacak; aynı zamanda kullanıcı doğrulama, alarm üretimi, trafik izleme ve geçmiş kayıtların görüntülenmesi gibi işlevleri içeren bir arayüz ile desteklenecektir. Böylece sistemin, teknik analiz yapan bir mekanizma olmasının yanında, kullanıcı tarafından izlenebilir ve yönetilebilir bir yapıya sahip olması amaçlanmaktadır.

Bu proje, yalnızca saldırı tespit başarısını artırmayı değil, aynı zamanda SDN ortamında çalışmaya uygun, düşük gecikmeli ve uygulamaya dönük bir çözüm ortaya koymayı hedeflemektedir. Bu doğrultuda proje ile SYN Flood saldırılarının erken aşamada belirlenmesi, ağ yöneticilerine hızlı geri bildirim sağlanması ve saldırı tespit sürecinin daha sistematik bir biçimde yürütülmesi amaçlanmaktadır. Ayrıca bu çalışma, SDN güvenliği alanında geliştirilebilecek daha kapsamlı sistemler için temel oluşturabilecek bir prototip ortaya koymayı hedeflemektedir.

A.2.2 Konu Ve Kapsam

Bu projenin konusu, Yazılım Tanımlı Ağ (SDN) ortamlarında TCP SYN Flood saldırılarının tespitine yönelik bir saldırı tespit sistemi geliştirmektir. Çalışmada, SDN denetleyicisinden elde edilen akış verileri kullanılarak saldırı tespitine uygun bir analiz yapısı kurulacak ve bu yapı bir kullanıcı arayüzü ile desteklenecektir.

Proje kapsamında SDN ortamından elde edilen akış istatistiklerinin toplanması, bu verilerin analiz için uygun hale getirilmesi, belirlenen özniteliklerle bir sınıflandırma modelinin çalıştırılması ve tespit sonuçlarının arayüz üzerinden sunulması yer almaktadır. Ayrıca sistem içerisinde kullanıcı girişi, alarm görüntüleme, trafik takibi ve geçmiş kayıtların listelenmesi gibi temel işlevlerin bulunması planlanmaktadır.

Çalışma yalnızca TCP SYN Flood saldırı türü ile ilgili olacaktır. Farklı saldırı türlerinin aynı sistem içinde birlikte ele alınması bu projenin kapsamı dışındadır. Benzer şekilde, geliştirilecek yapı saldırıyı otomatik olarak engelleyen bir önleme sistemi değil, saldırıyı tespit etmeye odaklanan bir IDS olarak ele alınacaktır. Kullanılacak veriler, SDN denetleyicisinden doğrudan elde edilebilen akış bilgileri ile sınırlı tutulacak; paket içeriği incelemesine dayalı yöntemler çalışma kapsamına dahil edilmeyecektir.

Bu kapsam çerçevesinde proje, hem teknik olarak uygulanabilir hem de arayüz desteği sayesinde kullanılabilir bir SDN tabanlı SYN Flood tespit sistemi ortaya koymayı hedeflemektedir.

A.2.3 Literatür Özeti

SDN üzerinde gerçekleştirilen DDoS ve özellikle SYN Flood saldırılarının tespiti, son yıllardaki önemli güvenlik problemlerinden biridir. Bu bağlamda, Arvind ve Radhika (2023) çalışmasında, Ryu kontrolcüsü ve Mininet kullanılarak SDN ortamındaki saldırıları tespit etmek amacıyla bir XGBoost modeli geliştirilmiştir. Bu çalışmada, XGBoost algoritmasının Logistic Regression ve Naive Bayes gibi diğer makine öğrenmesi algoritmalarına kıyasla daha yüksek doğruluk oranları sunduğu kanıtlanmıştır.

Literatürdeki bir diğer güncel çalışma olan Ji et al. (2023) çalışmasında, kontrolcüye yönelik saldırıları tespit etmek için Bilgi Entropisi (Information Entropy) yöntemleriyle desteklenmiş XGBoost tabanlı bir makine öğrenmesi algoritması önerilmiş ve yüksek doğruluk oranına ulaşılmıştır. Lakin, makaledeki tespit sisteminin eğitimi ve tüm performans testleri yalnızca “CICDDoS2019” isimli önceden toplanmış statik veri

seti üzerinden gerçekleştirilmiştir. Ağdaki anlık trafik akışını dinlemek yerine çevrimdışı verileri analiz eden bu yaklaşım, gerçek dünya senaryolarındaki anlık trafik değişimlerini yansıtmakta yetersiz kalabilmektedir (Ji et al., 2023). Çalışmamızda modeli statik verilerle eğitip bırakmak yerine doğrudan canlı bir SDN kontrolcüsüne entegre ederek gerçek zamanlı trafik analizi yapmasıyla bu metodolojik eksikliği doldurması hedeflenmektedir.

Nisha Ahuja, Debajyoti Mukhopadhyay ve Gaurav Singal tarafından yapılan çalışmada, SDN ortamlarında DDoS saldırılarını tespit etmek için DNN ve CNN gibi derin öğrenme algoritmaları kullanılmıştır. Bu modeller, akış tabanlı özellikler üzerinden yüksek doğruluk sağlamaktadır. Ancak çalışmada kullanılan veri setlerinin çevrimdışı, önceden etiketlenmiş verilerden oluştuğu ve yöntemin gerçek zamanlı ağ ortamlarında test edilmediği görülmektedir (Ahuja et al., 2024).

Espinoza Godínez et al. (2021) ve Hirsi et al. (2022) çalışmalarında ise LSTM tabanlı derin öğrenme modelleri kullanılarak DDoS saldırılarının tespiti ele alınmıştır. Bu çalışmaların performans metriklerinde yüksek başarı oranları elde ettiği gözlemlenmiştir. Fakat bu sonuçlar, yukarıdaki çalışmada olduğu gibi çevrimdışı ortamlarda yapılan değerlendirmelerde elde edilmiştir (Espinoza Godínez et al., 2021; Hirsi et al., 2022).

A.3 Beklenen Fayda

A.3.1 Özgün Değer

Bu çalışmanın özgün değeri, SDN ortamında SYN Flood saldırı tespitini yalnızca yüksek doğruluk üreten bir sınıflandırma problemi olarak ele almamasından kaynaklanmaktadır. Literatürde yer alan bazı çalışmalar, örneğin Ahuja et al. (2024), derin öğrenme modellerinin SDN güvenliği bağlamında etkili sonuçlar verdiğini göstermektedir. Bu durum, derin öğrenme tabanlı bir yaklaşımın proje için güçlü bir teknik temel oluşturduğunu ortaya koymaktadır. Ancak mevcut projede hedef yalnızca model performansı değil, bu modelin gerçek zamanlı çalışabilen ve kullanıcıya doğrudan çıktı sunabilen bir IDS sistemi içinde kullanılmasıdır.

Benzer biçimde, Ji et al. (2023) çalışmasında yüksek doğruluk oranlarına ulaşılmış olsa da, değerlendirme süreci önceden toplanmış statik veri setleri üzerinden yürütülmüştür. Bu nedenle yöntem, canlı ağ trafiğindeki anlık değişimleri doğrudan yansıtmaya açısından sınırlı kalmaktadır. Bu proje ise saldırı tespitini çevrimdışı analizle sınırlamayıp, SDN denetleyicisinden alınan akış verileri üzerinden gerçek zamanlı olarak gerçekleştirmeyi hedeflemektedir. Bu yönüyle çalışma, literatürde sık görülen çevrimdışı değerlendirme yaklaşımından ayrılmaktadır (Ji et al., 2023).

Ayrıca Das et al. (2022) çalışması doğrudan SYN Flood saldırısına odaklanması bakımından önemli bir örnek olsa da, sistem daha çok arka planda çalışan bir tespit yapısı şeklinde ele alınmıştır. Bu projede ise saldırı tespit süreci, kullanıcı girişi, alarm gösterimi, log kaydı ve trafik özetlerinin sunulduğu bir arayüz ile desteklenmektedir. Böylece yalnızca saldırıyı belirleyen bir model değil, izlenebilir ve kullanılabilir bir sistem ortaya konulmaktadır.

Derin öğrenme temelli çalışmalarda, özellikle Espinoza Godínez et al. (2021) ve Hirsi et al. (2022) tarafından önerilen LSTM ve CNN-LSTM tabanlı modellerin yüksek başarı oranları sunduğu görülmektedir. Bununla birlikte bu modellerin çoğu çevrimdışı ortamlarda değerlendirilmiş olup, gerçek zamanlı kullanımda işlem maliyeti ve gecikme açısından daha ağır yapılar oluşturabilmektedir (Espinoza Godínez et al., 2021; Hirsi et al., 2022). Fakat elde edilen doğruluk oranları incelendiğinde, uygun yapı uygulandığında derin öğrenme tabanlı bir sistemin daha başarılı olacağına karar verilmiştir.

Sonuç olarak bu projenin özgün değeri, literatürdeki model odaklı çalışmalardan farklı olarak saldırı tespitini gerçek zamanlı veri akışı, kullanıcı arayüzü ve sistem bütünlüğü ile birlikte ele almasıdır. Bu yönüyle çalışma, yalnızca teknik açıdan değil, uygulama ve kullanım açısından da daha işlevsel bir IDS yapısı önermektedir.

A.3.2 Yaygın Etki/ Katma Değer

Bu projenin tamamlanmasıyla birlikte, SDN tabanlı ağlarda SYN Flood saldırılarının tespiti için uygulanabilir bir prototip ortaya konulmuş olacaktır. Geliştirilecek sistem, ağ güvenliği alanında çalışan araştırmacılar, öğrenciler ve uygulayıcılar için örnek bir çalışma niteliği taşıyabilir. Özellikle SDN güvenliği, gerçek zamanlı saldırı tespiti ve akış verisi analizi konularında yeni çalışmalar için temel oluşturması beklenmektedir.

Çalışmanın bir diğer katma değeri, geliştirilecek sistemin yalnızca akademik bir model olarak kalmayıp kullanıcı tarafından izlenebilir bir yapıya sahip olmasıdır. Bu sayede sistem, eğitim ve uygulamalı gösterim amaçlı ortamlarda da kullanılabilir; SDN tabanlı saldırı tespitinin nasıl çalıştığını somut biçimde gösterebilecektir. Ayrıca çalışma, ileride farklı saldırı türlerinin eklenmesi, önleme mekanizmalarının entegre edilmesi veya model yapısının geliştirilmesi gibi yeni özelliklerin eklenmesi için de bir temel sunacaktır.

A.4 Yöntem

Bu projede öncelikle SDN tabanlı bir test ortamı kurulacaktır. Bu ortamda denetleyici, anahtarlar ve istemci-sunucu yapıları oluşturularak normal trafik ile TCP SYN Flood saldırı trafiği gözlemlenebilir hale getirilecektir. Daha sonra SDN denetleyicisinden belirli zaman aralıklarında akış verileri toplanacaktır. Paket sayısı, bayt sayısı, akış süresi, akış sayısı, kaynak ve hedef adres bilgileri ile port bilgileri gibi veriler saldırı tespit sürecinde kullanılacak temel girdiler arasında yer alacaktır. Bu çalışmada, veri seti olarak "SDN-TCP-SYN ATTACK-DDOS DATASET" kullanılacaktır ve veri tabanı olarak da SQLite kullanılacaktır.

Toplanan veriler kapsamlı bir ön işleme aşamasından geçirilerek canlı SDN sistemi için uygun hale getirilecektir. Bu aşamada veri sızıntısına yol açabilecek kimlik kolonları ve canlı ağ anahtarlarından anlık olarak elde edilmesi zor olan karmaşık istatistikler veri setinden çıkarılacaktır. 'Saniyedeki Paket Sayısı' ve 'Ortalama Paket Boyutu' gibi canlı ağda gecikmesiz hesaplanabilecek en etkili öznitelikler seçilerek model hafifletilecektir. Ardından, derin öğrenme modellerinin SDN kontrolcüsü üzerinde yaratacağı donanım yükü ve karar gecikmesi göz önüne alınarak; derin öğrenme modeli geliştirilecek ve normal trafik ile SYN Flood trafiği arasındaki sınıflandırma işlemi gerçekleştirilecektir.

Modelin geliştirilmesinden sonra sistem performansı çeşitli ölçütlerle değerlendirilecektir. Bu kapsamda doğruluk, precision, recall, F1-score ve gecikme süresi temel değerlendirme ölçütleri olarak ele alınacaktır. Son aşamada ise tespit sonuçlarını kullanıcıya sunacak arayüz geliştirilecek; kullanıcı girişi, alarm gösterimi, trafik görüntüleme ve geçmiş kayıtların saklanması gibi bileşenler sisteme entegre edilecektir. Böylece veri toplama, analiz, karar verme ve sonuçların kullanıcıya sunulması tek bir yapı içinde birleştirilmiş olacaktır.

A.5 Araştırma Yöntemi

Bu proje, SDN ve yapay zeka tabanlı siber güvenlik çalışmaları için şu teknik imkanları sunmaktadır:

- Gerçekçi Ağ Simülasyonu: Mininet ve OS-Ken (Ryu) kullanarak gerçek bir ağ gibi çalışan test ortamları kurma imkanı vardır.
- Doğrudan Veri Erişimi: Trafik verilerini dışarıdan bir izleme cihazına gerek duymadan, doğrudan denetleyici (controller) üzerinden çekebiliyoruz.
- Özel Veri Seti Üretimi: SYN Flood saldırısına has "SYN/ACK oranı" ve "yarım açık bağlantı tahmini" gibi kritik güvenlik verilerini hesaplama şansımız mevcuttur.
- Model Karşılaştırma: XGBoost gibi hızlı modeller ile MLP gibi derin öğrenme modellerini doğruluk

ve hız açısından kıyaslama imkanı bulunmaktadır.

- Uçtan Uca Sistem Entegrasyonu: Veri toplama, yapay zeka tahmini, veritabanı kaydı ve kullanıcı panelinin bir arada çalıştığı tam bir sistem geliştirme fırsatı sunulmaktadır.
- Hibrit Tespit Analizi: Sadece yapay zekaya bağlı kalmayıp, hızlı kural filtreleri ile modellerin beraber çalıştığı bir güvenlik yapısını test etme olanağı sağlar.

A.6 Çalışma Takvimi

A.6.1 İş Zaman Çizelgesi

İş Tanımı	Çalışacak Kişiler	Başlama Tarihi	İş Süreleri (gün)	Bitiş Tarihi	P/G	Sub W1	Sub W2	Sub W3	Sub W4	Mar W1	Mar W2	Mar W3	Mar W4	Mar W5	Nis W1	Nis W2	Nis W3	Nis W4	May W1	May W2	May W3	May W4
Literatür Taraması, Senaryo ve Bileşen Belirleme	Hepsi	01.02.2026	21	22.02.2026	Planlanan																	
SDN Test Ortamı ve Veri Toplama Altyapısı Kurulumu	Taha Aydın, Atakan Berber, Umut Öztürk	01.02.2026	28	28.02.2026	Gerçekleşen																	
Veri Toplama, Ön İşleme ve Öznitelik Belirleme	Atakan Berber, Umut Öztürk	15.02.2026	21	15.03.2026	Gerçekleşen																	
Saldırı Tespit Modeli Geliştirme ve Eğitimi	Taha Aydın, Eren Eroğlu	12.04.2026	28	10.05.2026	Planlanan																	
Model Performans Değerlendirmesi (Kritik Eşik)	Taha Aydın, Eren Eroğlu	26.04.2026	28	24.05.2026	Gerçekleşen																	
Kullanıcı Doğrulama ve Arka Uç (Auth/API) Geliştirme	Taha Aydın, Umut Öztürk	29.03.2026	21	12.04.2026	Planlanan																	
Alarm Gösterimi ve Trafik Takibi Modülleri	Umut Öztürk, Eren Eroğlu	29.03.2026	14	12.04.2026	Gerçekleşen																	
Kayıt Görüntüleme ve Veritabanı Entegrasyonu	Eren Eroğlu, Atakan Berber	12.04.2026	7	03.05.2026	Planlanan																	
Sistem Bütünsel Testi ve Hata Giderme	Taha Aydın, Umut Öztürk, Eren Eroğlu	26.04.2026	14	17.05.2026	Planlanan																	
Final Raporlama ve Dokümantasyon Süreci	Hepsi	03.05.2026	10	27.05.2026	Planlanan																	

Figür 1. İş zaman çizelgesi

Çalışmanın ilk aşamasında literatür taraması yapılarak proje kapsamı netleştirilecek, kullanılacak saldırı senaryosu ve sistem bileşenleri belirlenecektir. İkinci aşamada SDN test ortamı kurulacak ve denetleyiciden akış verisi toplanmasına yönelik altyapı hazırlanacaktır. Üçüncü aşamada veri toplama, ön işleme ve öznitelik belirleme çalışmaları yürütülecektir.

Sonraki aşamada saldırı tespit modeli geliştirilecek, eğitilecek ve performans değerlendirmesi yapılacaktır. Model sonuçlarının yeterli bulunması halinde arayüz geliştirme sürecine geçilecek ve kullanıcı doğrulama, alarm gösterimi, trafik takibi ve kayıt görüntüleme modülleri tamamlanacaktır. Son aşamada sistem bütün olarak test edilecek, eksikler giderilecek ve raporlama süreci tamamlanacaktır.

A.6.2 Kim Ne İş Yapacak

YAPILACAK İŞ	TAHA AYDIN	UMUT ÖZTÜRK	ATAKAN BERBER	EREN EROĞLU
Proje gereksinimlerinin ve sistem mimarisinin belirlenmesi	X	X	X	X
SDN test ortamının kurulması	X		X	
OS-Ken denetleyicisinden veri toplama altyapısının hazırlanması		X	X	
Trafik üretimi ve veri seti kontrolü		X	X	
Veri ön işleme ve öznitelik hazırlama			X	
Saldırı tespit modelinin geliştirilmesi	X			X
Python backend servisinin geliştirilmesi	X	X	X	
SQLite veritabanı yapısının kurulması				X
React tabanlı kullanıcı arayüzünün geliştirilmesi		X		X
Sistem entegrasyonu	X	X		X
Test ve performans değerlendirme	X			X
Dokümantasyon, raporlama ve sunum hazırlığı	X	X	X	X

Figür 2. Kişi ve yapılan iş listesi

B. ANALİZ

B.1 Sistem Gereksinimlerini Ortaya Çıkarma Yöntem ve Teknikleri

B.1.1 Yüz Yüze Görüşme

Danışmanımız Dr. Öğr. Üyesi İlker Özçelik ile yapılan görüşmede, projenin temel amacı, kapsamı ve uygulanabilirliği üzerine detaylı bir değerlendirme yapılmıştır. Görüşmelerde SDN ortamında TCP SYN Flood saldırılarını tespit edebilen gerçek zamanlı çalışan bir saldırı tespit sistemi geliştirme fikrinde mutabık kalındı. Sistemin teknik açıdan uygulanabilir ve raporlanabilir bir proje yapısına sahip olması gerektiği üzerinde durulmuştur.

Çalışmanın odak noktası, genel DDoS saldırıları yerine SYN Flood saldırılarını incelenmesi olarak kararlaştırılmıştır. Veriseti, controller üzerinden doğrudan çekilebilen veriler doğrultusunda analiz edilmiştir. Projede kullanılacak araçlar belirlenmiş, bu araçların nasıl kullanıldığına yönelik araştırmalar yapılmıştır.

B.1.2 Yazılı Basılı Belge İncelemesi

Sistem gereksinimlerini belirlemek amacıyla proje konusu ile doğrudan ilişkili yazılı ve basılı kaynaklar incelenmiştir. Bu süreçte, SDN mimarisi, SYN Flood saldırıları, SDN tabanlı saldırı tespit sistemleri, akış verisi analizi ve makine öğrenmesi / derin öğrenme tabanlı tespit yaklaşımları üzerine yayımlanan akademik çalışmalar incelenmiştir. Bu sürecin sonunda; projenin teknik gereksinimleri, kullanılabilecek veri türleri ve mevcut çalışmaların hangi yönleri vurguladığı daha açık bir biçimde ortaya konulmuştur. İncelenen başlıca kaynaklar aşağıda özetlenmiştir:

- **SDN ve OpenFlow dokümantasyonları:**

SDN mimarisinin çalışma mantığı, denetleyici-veri düzlemi ayrımı, OpenFlow tabanlı iletişim yapısı ve denetleyiciden elde edilebilecek akış istatistikleri incelenmiştir. Bu inceleme sonucunda, paket sayısı, bayt sayısı, akış süresi, kaynak/hedef adres bilgileri ve port bilgileri gibi verilerin saldırı tespiti açısından kullanılabilecek temel girdiler olduğu görülmüştür. Bununla birlikte, denetleyiciden doğrudan alınamayan bazı davranışsal özelliklerin sonradan türetilmesi gerektiği de anlaşılmıştır.

- **SYN Flood ve SDN güvenliği üzerine akademik çalışmalar:**

SYN Flood saldırısının özellikle SDN ortamında yalnızca hedef sistemi değil, denetleyici yükünü de etkileyebildiğini ele alan çalışmalar incelenmiştir. Yapılan değerlendirmelerde, mevcut çalışmaların bir kısmının genel DDoS saldırılarına odaklandığı, bir kısmının ise çevrimdışı veri kümeleri üzerinde sonuç ürettiği görülmüştür. Bu durum, doğrudan SYN Flood saldırısına odaklanan ve gerçek zamanlı çalışabilecek sistem tasarımlarının eksikliği göstermiştir.

- **Makine öğrenmesi ve derin öğrenme tabanlı IDS çalışmaları:**

Akış verileri kullanılarak geliştirilen saldırı tespit modelleri incelenmiş; özellikle CNN-LSTM, çevrimiçi öğrenme tabanlı sınıflandırıcılar, Random Forest ve XGBoost gibi yöntemlerin kullanıldığı çalışmalar değerlendirilmiştir. Bu incelemeler sonucunda, bazı modellerin yüksek doğruluk sağladığı ancak gecikme ve sistem yükü açısından her zaman uygun olmadığı görülmüştür. Bu nedenle yalnızca doğruluk oranına değil, gerçek zamanlı uygulanabilirliğin de dikkat edilmesi gerektiği sonucuna varılmıştır.

- **Arayüz ve sistem bütünlüğü açısından değerlendirilen kaynaklar:**

İncelenen çalışmaların önemli bir bölümünde saldırı tespiti modelinin ön planda olduğu, ancak kullanıcı girişi, alarm yönetimi, trafik izleme ve geçmiş kayıtların görüntülenmesi gibi bileşenlerin çoğu zaman sistemin parçası olarak ele alınmadığı görülmüştür. Bu durum, geliştirilecek projenin yalnızca model odaklı değil, aynı zamanda kullanıcıya sunulan işlevler açısından da bütünlük bir

yapı taşınması gerektiğini göstermiştir.

B.1.3 Anket

Siber güvenlik veya ağ yönetimi (Networking) alanındaki bilgi ve tecrübe seviyenizi nasıl tanımlarsınız?

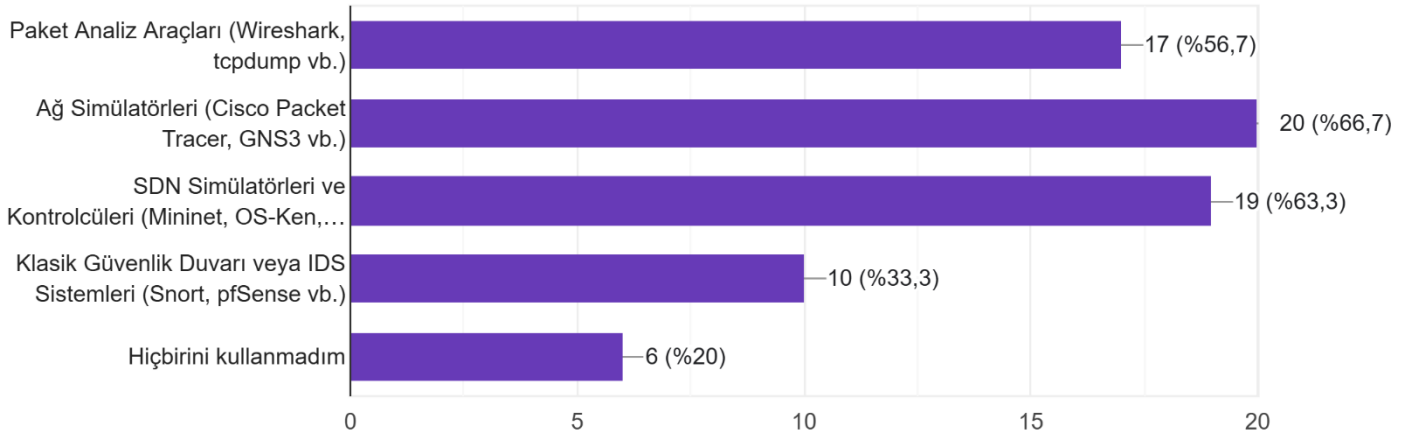
30 yanıt



Figür 3. Siber güvenlik bilgi düzeyi anket sonuçları

Bugüne kadar ağ analizi, simülasyonu veya güvenliği için aşağıdaki araç gruplarından hangilerini deneyimleme fırsatınız oldu?

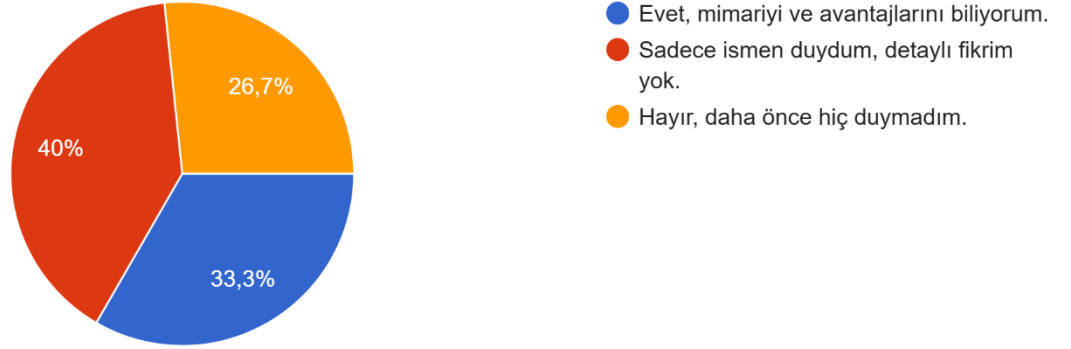
30 yanıt



Figür 4. Ağ alanındaki deneyim anketi sonuçları

SDN (Yazılım Tanımlı Ağlar - Software Defined Networking) mimarisinin temel mantığı (Kontrol ve Veri düzleminin ayrılması) hakkında bilgi sahibi misiniz?

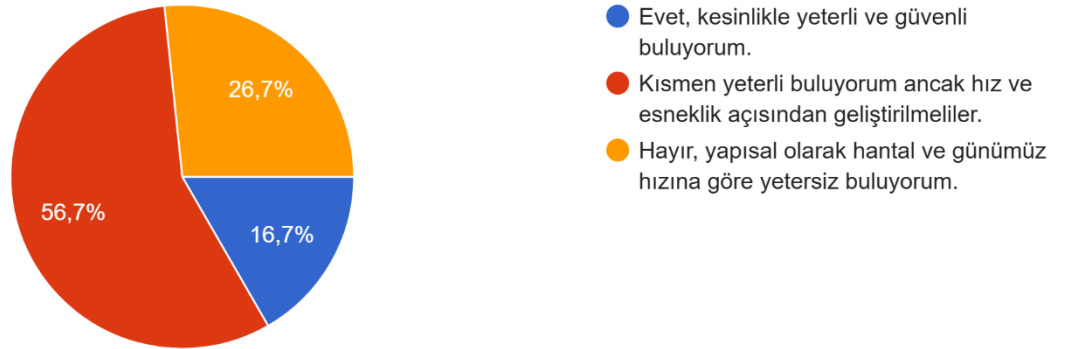
30 yanıt



Figür 5. SDN bilgi düzeyi anket sonuçları

Geleneksel ağ altyapılarında çalışan "Donanıma Bağımlı Klasik IDS/IPS Sistemlerini", günümüzün yüksek hacimli DDoS ve SYN Flood saldırılarına karşı yeterli buluyor musunuz?

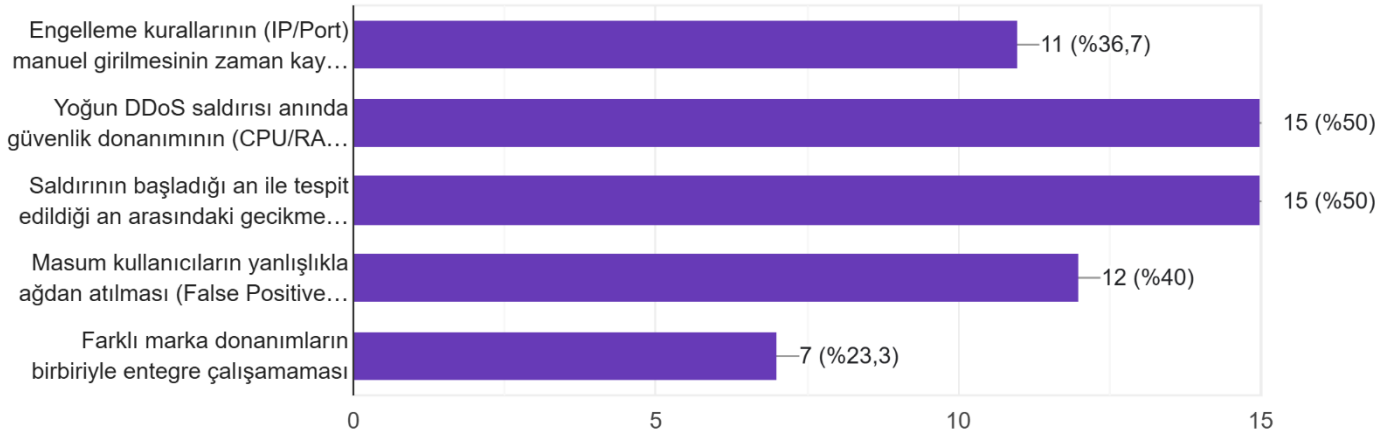
30 yanıt



Figür 6. DDoS ve SYN Flood hakkındaki anket sonuçları

Geleneksel ağ güvenlik sistemlerini (Firewall vb.) yönetirken veya incelerken sektörde en çok hangi zorlukların yaşandığını düşünüyorsunuz?

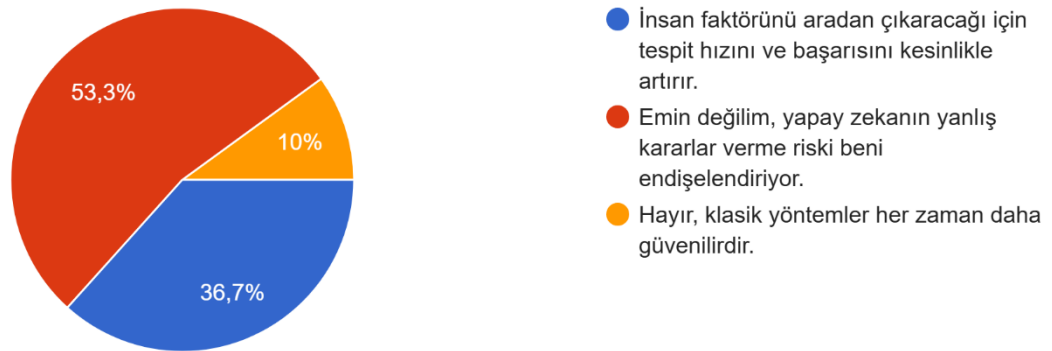
30 yanıt



Figür 7. Geleneksel ağ hakkındaki anket sonuçları

Ağ güvenliğinde tespiti statik kurallar yerine "Makine Öğrenmesi (Yapay Zeka)" modellerine bırakmanın (otonomlaştırmanın) başarıyı nasıl etkileyeceğini düşünüyorsunuz?

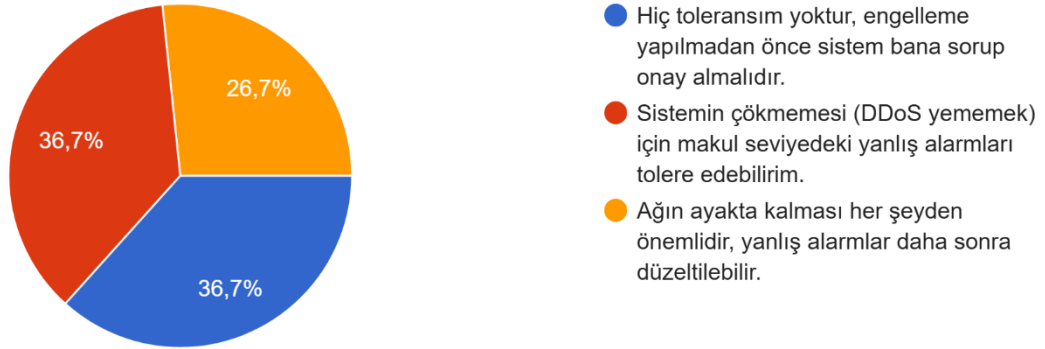
30 yanıt



Figür 8. Makine öğrenmesi - ağ güvenliği ilişkisi hakkındaki anket sonuçları

Yapay zeka destekli otonom bir güvenlik sisteminde, "yanlış alarm" (masum bir kullanıcıyı saldırgan sanıp engelleme) riskine karşı bir ağ yöneticisi olarak yaklaşımınız ne olurdu?

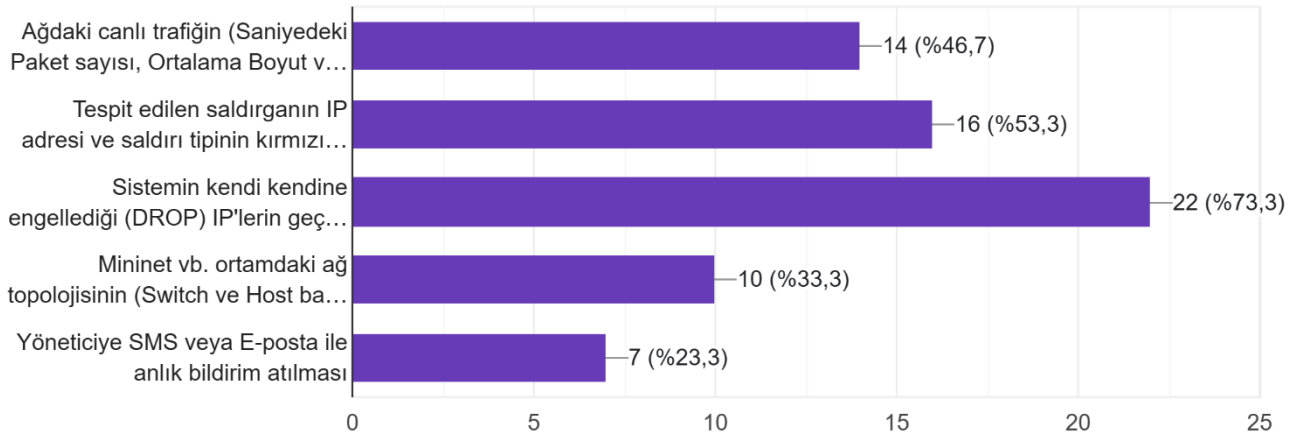
30 yanıt



Figür 9. Yapay zekanın yanlış atak tespiti hakkındaki anket sonuçları

Otonom bir SDN güvenlik kontrol panelinde (Web Dashboard) sizin için "olmazsa olmaz" özellikler hangileridir?

30 yanıt



Figür 10. SDN kontrol uygulamasının grafiksel kullanıcı arayüzünde olması gerekenler hakkındaki anket sonuçları

SDN mimarisi üzerinde çalışan, milisaniye hızında tespiti yapıp saldırgan IP'yi anında switch üzerinden engelleyebilen böyle bir projeyi kendi ağınıza test etmek ister miydiniz?

30 yanıt



Figür 11. SDN mimarisinde atak tespitinin yapılabilmesi hakkındaki anket sonuçları

Gerçekleştirilen anket çalışmasının sonuçları analiz edildiğinde, proje kapsamında geliştirilen SDN tabanlı otonom güvenlik mimarisinin sektördeki kritik bir darboğazı hedef aldığı açıkça görülmektedir. Ankete katılan ve ağırlıklı olarak orta/temel düzey ağ tecrübesine sahip katılımcıların %95'lik bir çoğunluğu (Kısmen %60, Tamamen Yetersiz %35), geleneksel donanım bağımlı IDS/IPS sistemlerinin günümüzdeki yüksek hacimli DDoS ve SYN Flood saldırılarına karşı yetersiz kaldığını belirtmiştir. Ağların çökmesine neden olan en büyük güvenlik zafiyetleri olarak **"saldırıların çok geç fark edilmesi (gecikme/latency)"** ve **"güvenlik donanımlarının aşırı yük altında kilitlenmesi"** seçeneklerinin öne çıkması, projemizin temel tasarım motivasyonunu doğrudan doğrulamaktadır.

Çözüm yaklaşımı olarak katılımcıların %60'ı statik kurallar yerine Makine Öğrenmesi (Yapay Zeka) modellerinin kullanımının tespit hızını ve başarıyı artıracaklarını savunmuştur. Bununla birlikte, %30'luk bir kesimin yapay zekanın üretebileceği "yanlış alarm (False Positive)" riskinden endişe duyması son derece dikkat çekicidir. Ankette ortaya çıkan bu endişe; sistemimizde neden rastgele bir eşik değeri kullanılmadığının, modelin ROC eğrisi üzerinden neden hassas bir optimizasyondan geçirildiğinin ve kontrolcüyü (OS-Ken) yormamak adına veri setindeki 84 özellikten neden sadece en hafif 5 özneliğin (feature) seçildiğinin en büyük kanıtıdır. Sonuç olarak bu veriler, geleneksel hantal donanımlara kıyasla milisaniye hızında tepki verebilen, hafifletilmiş yapay zeka destekli bir SDN çözümünün hem akademik hem de pratik düzeyde ne kadar gerekli bir mühendislik yaklaşımı olduğunu sayısal olarak ispatlamaktadır.

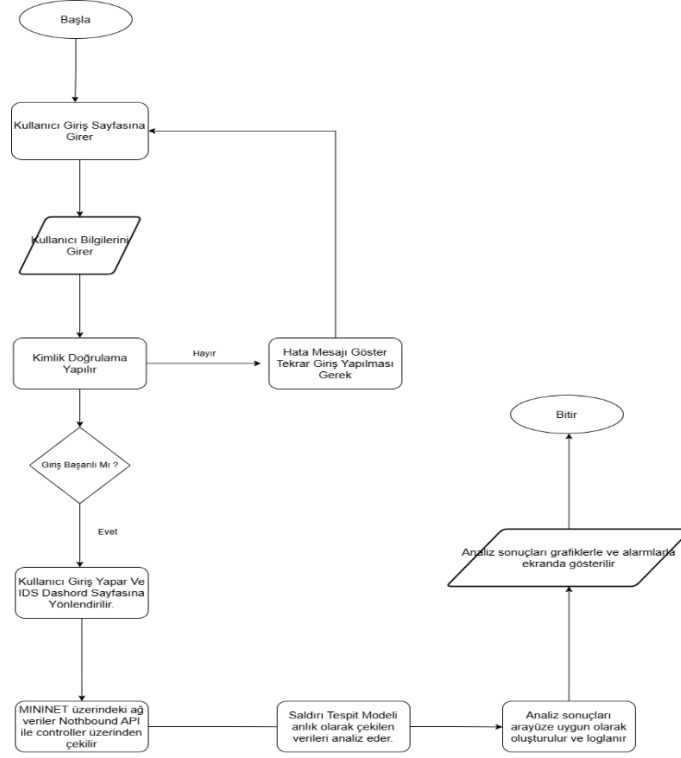
B.1.4 Gözlem

Proje kapsamında yapılan literatür taraması, danışman görüşmeleri, teknik incelemeler ve kurulan test ortamında üzerinde yapılan gözlemler sonucunda; projenin uygulanabilir olduğuna karar verilmiştir. Yapılan incelemeler ve ihtiyaç analizleri sonucunda sistemin kullanıcı arayüzü, alarm yapısı ve kayıt takibi gibi bileşenlerle desteklenmesinin projeyi daha işlevsel hale getireceği kararlaştırılmıştır. Ubuntu üzerinde kurulacak Mininet tabanlı SDN ortamı ile OS-Ken denetleyicisinin bu proje için uygun bir test zemini sunduğu, Python tabanlı backend, SQLite veritabanı ve React tabanlı arayüz kullanımının da hem geliştirme sürecini yönetilebilir kılacağı hem de işlevsel bir prototip oluşturulmasına katkı sağlayacağı değerlendirilmiştir. Tüm bu gözlemler doğrultusunda, projenin yalnızca akademik açıdan değil, teknik uygulama ve sistem bütünlüğü açısından da sürdürülebilir ve geliştirilebilir bir yapıya sahip olduğu sonucuna varılmıştır.

B.1.5 Veri Akış Şemaları

B.1.5.1 Kavramsal Veri Akış Şeması

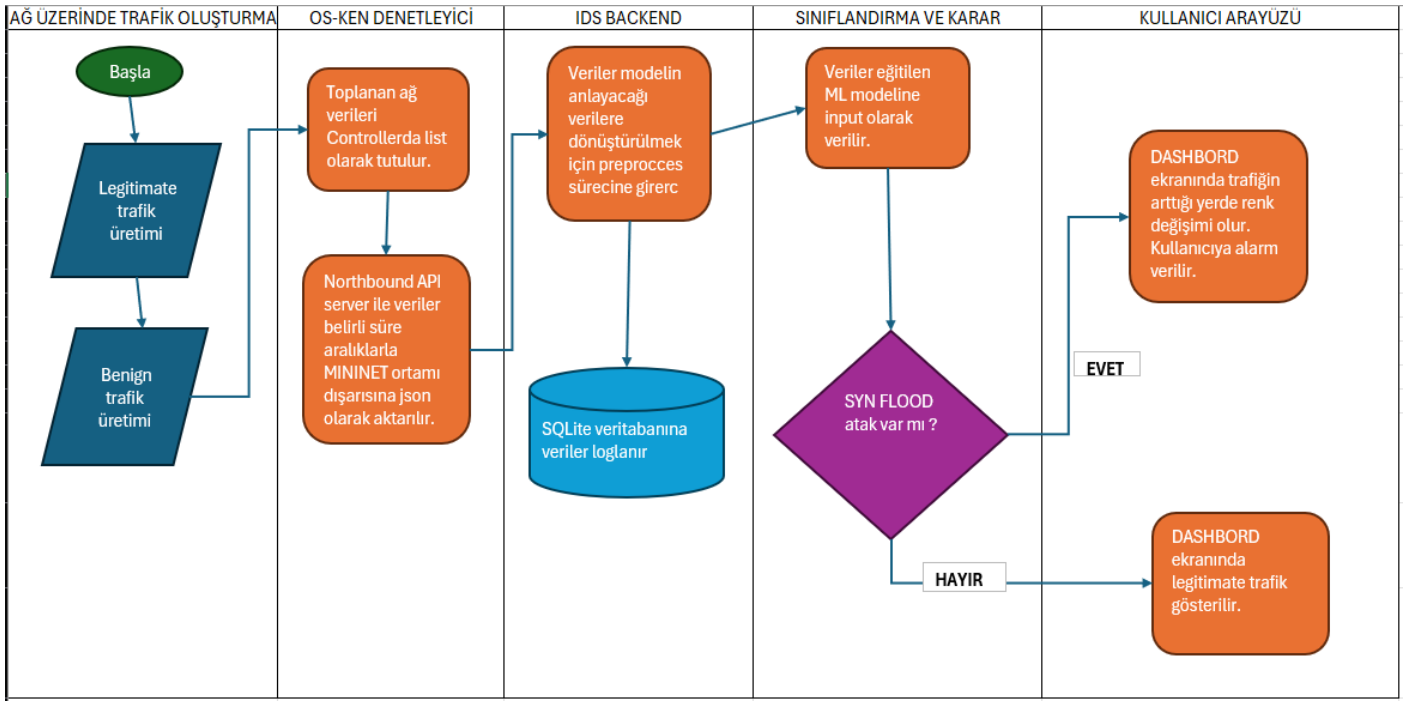
Kavramsal Veri Akış Şeması, elıştırilecek IDS sisteminin genel çalışma mantığını ve veri akışını en üst düzeyde gösteren bir yapıdır. Bu şema, SDN ortamında oluşan ağ trafiğinin Mininet üzerinden üretilmesiyle başlayıp, OS-Ken denetleyicisi tarafından akış verilerinin toplanması, bu verilerin backend tarafında işlenmesi, saldırı tespit modeline aktarılması ve elde edilen sonuçların kullanıcı arayüzü üzerinden sunulması sürecini temsil etmektedir. Kavramsal şema; ağ ortamı, denetleyici, veri işleme katmanı, saldırı tespit mekanizması, veritabanı ve kullanıcı arayüzü arasındaki temel ilişkileri göstermekte, böylece sistemin bütünsel yapısının anlaşılmasına yardımcı olmaktadır.



Figür 12. Kavramsal veri akış şeması

B.1.5.2 Mantıksal Veri Akış Şeması

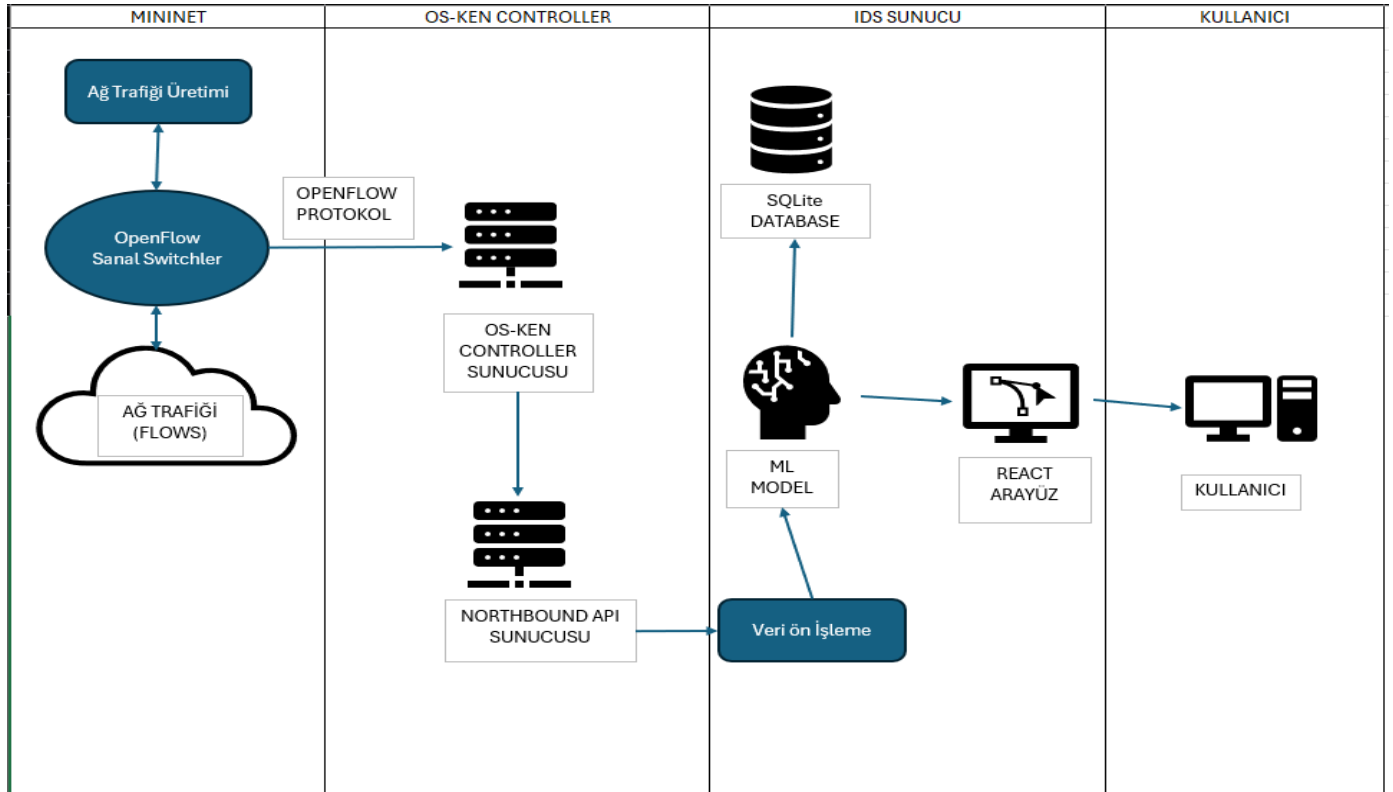
Bu çalışmada geliştirilen SDN tabanlı IDS sisteminde mantıksal veri akışı, ağ trafiğinin oluşmasıyla başlayıp bu trafiğe ait akış bilgilerinin toplanması, işlenmesi, sınıflandırılması ve sonuçların kullanıcı arayüzü üzerinden sunulması şeklinde ilerlemektedir. Sistem, gerçek zamanlı ağ trafiği üzerinde TCP SYN Flood saldırılarını tespit etmeyi amaçlamakta olup, saldırı sonrası otomatik önleme veya engelleme mekanizması içermemektedir.



Figür 13. İş zaman çizelgesi

B.1.5.3 Fiziksel Veri Akış Şeması

Fiziksel Veri Akış Şeması, geliştirilen IDS sisteminde verinin hangi fiziksel ve yazılımsal bileşenler arasında nasıl aktığını gösteren yapıdır. Bu şema, Ubuntu işletim sistemi üzerinde kurulu Mininet tabanlı SDN ortamında üretilen ağ trafiğinin OpenFlow anahtarları üzerinden OS-Ken denetleyicisine ulaşmasını, denetleyici tarafından elde edilen akış istatistiklerinin Python tabanlı backend katmanına aktarılmasını, burada işlenerek saldırı tespit modeline gönderilmesini ve elde edilen sonuçların SQLite veritabanına kaydedilip React tabanlı kullanıcı arayüzüne sunulmasını açıklamaktadır.



Figür 14. Fiziksel veri akış şeması

B.1.6. Olay Tabloları, Durum Formları, İşlevsel Analiz Raporu, İş Akış Şeması

B.1.6.1 Olay Tabloları

Aşağıdaki olay tablosu, geliştirilen SDN tabanlı IDS sistemi kapsamında gerçekleşebilecek temel olayları ve bu olaylara karşı sistemin verdiği yanıtları özetlemektedir. Tabloda her bir olayın kaynağı, sistem tarafından nasıl işlendiği ve süreç sonunda hangi çıktının oluştuğu gösterilmiştir. Bu yapı sayesinde kullanıcı etkileşiminden denetleyici üzerinden veri toplanmasına, saldırı tespitinden sonuçların arayüzde sunulmasına kadar sistemin temel işleyişi açık ve düzenli biçimde ortaya konulmaktadır. Olay tabloları, analiz ve tasarım aşamalarında gereksinimlerin netleştirilmesi ve sistem davranışlarının önceden tanımlanması açısından önemli bir rol oynamaktadır.

Olay Id	Olay Adı	Olay Tanımı	Kaynak	Sistem Yanıtı	Çıktı
1	Kullanıcı Girişi	Kullanıcı, giriş ekranında username ve password bilgileriyle giriş yapar.	Kullanıcı + Sistem	Girilen bilgiler veri tabanından doğrulanır. Bilgiler doğruysa giriş yapılır, hatalıysa uyarı mesajı gösterilir.	Başarılı girişte kullanıcı panele yönlendirilir; başarısız girişte uyarı mesajı oluşur.
2	Panelin Açılması	Kullanıcı giriş yaptıktan sonra ana panel görüntülenir	Kullanıcı	Sistem; mevcut saldırı kayıtlarını, ağın son durumu hakkındaki bilgilerini ve trafik özetini backend üzerinden çekerek arayüzde gösterir.	Kullanıcıya ana panel ekranı sunulur.
3	Controller bağlantısının kontrolü	Sistemi OS-Ken controller ile bağlantının aktif olup olmadığını kontrol eder.	Sistem	Controller'dan veri gelip gelmediği kontrol edilir. Bağlantı varsa süreç devam eder, hata varsa uyarı mesajı verilir ve loglara yansıtılır.	"Bağlı / Bağlantı yok" durumu arayüze ve loglara yansıtılır.
4	Akış Verilerinin Toplanması	SDN ortamında oluşan trafik akışlarına ait istatistikler belirli aralıklarla	OS-Ken Denetleyicisi	Sistem; paket sayısı, bayt sayısı, akış süresi, kaynak-hedef IP, port ve protokol	Ham akış verileri backend katmanına aktarılır.

		denetleyiciden alınır.		gibi controller üzerinden doğrudan alınabilecek verileri toplar.	
5	Veri Ön İşleme	Toplanan flow verileri modele verilmeden önce düzenlenir.	Sistem	Eksik veya tutarsız kayıtlar ayıklanır, gerekli dönüşümler uygulanır ve veri model girişine uygun hale getirilir.	Temizlenmiş ve işlenmiş veri kümesi elde edilir.
6	Özellik Çıkarma	Ön işlenmiş verilerden saldırı tespiti için kullanılacak özellikler hazırlanır.	Sistem	SYN yoğunluğu, paket oranı, akış davranışı gibi saldırıyı ayırt etmeye yardımcı olacak özellikler hesaplanır.	Model girişine uygun özellik vektörü oluşturulur.
7	Trafiğin Sınıflandırılması	İşlenen veri, eğitilmiş saldırı tespit modeline gönderilir.	IDS Modeli	Model, ilgili akışın normal trafik mi yoksa saldırı davranışı mı içerdiğine karar verir.	“Normal” veya “Saldırı” etiketi üretilir.
8	Alarm Kaydı Oluşturulması	Model çıktısı saldırı yönünde ise sistem bu durumu kayıt altına alır.	Sistem	Tespit edilen saldırı bilgisi zaman damgası ile birlikte veritabanına kaydedilir.	Alarm kaydı ve olay geçmişi oluşturulur.
9	Sonucun Kullanıcıya Gösterilmesi	Sınıflandırma sonucu arayüzde kullanıcıya sunulur.	Sistem + Kullanıcı Arayüzü	Backend, sonucu React arayüzüne iletir. Arayüz saldırı, normal trafik, alarm ve log bilgilerini uygun biçimde gösterir.	Kullanıcı saldırı uyarısını, log kaydını ve ilgili özet bilgileri ekranda görür.
10	Geçmiş Kayıtların Görüntülenmesi	Kullanıcı önceki saldırı kayıtlarını ve logları incelemek ister.	Kullanıcı	Sistem veritabanındaki kayıtları sorgular ve liste halinde kullanıcıya sunar.	Geçmiş alarm ve kayıt ekranı görüntülenir.

11	Trafik/Saldırı Detayı Görüntüleme	Kullanıcı belirli bir olayın detayını incelemek ister.	Kullanıcı	Seçilen kayda ait zaman, trafik bilgisi, sınıflandırma sonucu ve ilgili özet detaylar getirilir.	Olay detay ekranı açılır.
12	Kullanıcı Çıkışı	Kullanıcı aktif oturumu sonlandırmak ister.	Kullanıcı + Sistem	Sistem oturumu kapatır ve kullanıcıyı giriş ekranına yönlendirir.	Oturum sonlandırılır, kullanıcı çıkış yapmış olur.

B.1.6.2 Durum Formları

Aşağıda geliştirilen SDN tabanlı IDS sistemine ait temel durum formları verilmiştir. Bu formlar, sistemin belirli durumlar karşısında nasıl davrandığını, hangi girdileri kullandığını, hangi işlemleri gerçekleştirdiğini ve süreç sonunda hangi çıktıları ürettiğini göstermektedir. Böylece kullanıcı işlemleri, veri toplama süreci, saldırı tespiti ve sonuçların sunulması gibi temel sistem davranışları daha açık biçimde tanımlanmış olmaktadır.

1. Kullanıcı Girişi

Durum Adı	Kullanıcı Girişi
Amaç	Sisteme yalnızca yetkili kullanıcıların erişmesini sağlamak
Girdi	Kullanıcı adı ve parola
Koşul	Kullanıcı giriş ekranında geçerli bilgileri girmelidir
İşlem	Girilen bilgiler backend üzerinden doğrulanır
Çıktı	Başarılıysa kullanıcı panele yönlendirilir, başarısızsa hata mesajı gösterilir

2. Controller Bağlantısının Kontrolü

Durum Adı	Controller Bağlantısının Kontrolü
Amaç	OS-Ken denetleyicisinin aktif olup olmadığını belirlemek
Girdi	Denetleyiciye gönderilen bağlantı isteği veya durum sorgusu
Koşul	SDN ortamı ve denetleyici çalışıyor olmalıdır
İşlem	Sistem controller'dan veri almayı dener ve bağlantı durumunu kontrol eder
Çıktı	Controller bağlıysa veri toplama süreci başlar, değilse bağlantı hatası kaydedilir ve uyarı verilir

3. Akış Verilerinin Toplanması

Durum Adı	Akış Verilerinin Toplanması
Amaç	SDN ortamındaki trafik akışlarına ait temel istatistikleri elde etmek
Girdi	OpenFlow switch'lerden gelen akış verileri
Koşul	Controller bağlantısı aktif olmalıdır
İşlem	Paket sayısı, bayt sayısı, akış süresi, IP, port ve protokol bilgileri toplanır
Çıktı	Ham akış verileri backend katmanına aktarılır

4. Veri Ön İşleme

Durum Adı	Veri Ön İşleme
Amaç	Toplanan verileri modelin kullanabileceği yapıya dönüştürmek
Girdi	Ham akış verileri
Koşul	Veri toplama işlemi başarıyla tamamlanmış olmalıdır
İşlem	Eksik kayıtların kontrolü, veri temizleme ve uygun formata dönüştürme işlemleri gerçekleştirilir
Çıktı	Temizlenmiş ve işlenmiş veri elde edilir

5. Özellik Çıkarma

Durum Adı	Özellik Çıkarma
Amaç	Saldırı tespitinde kullanılacak öznitelikleri oluşturmak
Girdi	Ön işlenmiş akış verileri
Koşul	Veri modeli besleyecek yapıya getirilmiş olmalıdır
İşlem	Paket oranı, SYN davranışı ve akış yoğunluğu gibi özellikler hazırlanır
Çıktı	Model girişine uygun öznitelik seti oluşturulur

6. Trafik Sınıflandırma

Durum Adı	Trafik Sınıflandırma
Amaç	Ağ trafiğinin normal mi yoksa saldırı mı olduğunu belirlemek
Girdi	Özellik çıkarımı tamamlanmış veri
Koşul	Model sisteme entegre edilmiş ve çalışır durumda olmalıdır
İşlem	Veri saldırı tespit modeline gönderilir ve sınıflandırma sonucu elde edilir
Çıktı	Trafik “normal” veya “saldırı” olarak etiketlenir

7. Alarm Kaydı Oluşturma

Durum Adı	Alarm Kaydı Oluşturma
Amaç	Tespit edilen saldırı durumlarını kayıt altına almak
Girdi	Model tarafından üretilen “saldırı” sonucu
Koşul	Sınıflandırma sonucu saldırı olmalıdır
İşlem	Saldırıya ait zaman bilgisi ve özet kayıt veritabanına yazılır
Çıktı	Alarm kaydı ve geçmiş olay verisi oluşturulur

8. Sonucun Arayüzde Gösterilmesi

Durum Adı	Sonucun Arayüzde Gösterilmesi
Amaç	Tespit sonucunu kullanıcıya anlaşılır biçimde sunmak
Girdi	Sınıflandırma sonucu ve alarm/log kayıtları
Koşul	Kullanıcı sisteme giriş yapmış olmalıdır
İşlem	Backend, sonucu React arayüzüne iletir; arayüz bunu uygun ekran bileşenleri ile gösterir
Çıktı	Kullanıcı alarm, log ve trafik özetini ekranda görür

9. Geçmiş Kayıtların Görüntülenmesi

Durum Adı	Geçmiş Kayıtların Görüntülenmesi
Amaç	Önceden tespit edilen olayların incelenmesini sağlamak
Girdi	Kullanıcının kayıt sorgulama isteği
Koşul	Veritabanında kayıt bulunmalıdır
İşlem	SQLite (yeterli olmayabilir, denemelerden sonra daha büyük bir veritabanı ihtiyacı doğabilir) veritabanından geçmiş kayıtlar çekilir ve liste halinde sunulur
Çıktı	Kullanıcı geçmiş saldırı kayıtlarını görüntüler

10. Kullanıcı Çıkışı

Durum Adı	Kullanıcı Çıkışı
Amaç	Aktif oturumu güvenli biçimde sonlandırmak
Girdi	Kullanıcının çıkış isteği
Koşul	Kullanıcı sisteme giriş yapmış olmalıdır
İşlem	Sistem oturumu kapatır ve giriş ekranına yönlendirir
Çıktı	Kullanıcı sistemden çıkış yapmış olur

B.1.6.3 İşlevsel Analiz Raporu

Aşağıdaki işlevsel analiz raporu, SDN tabanlı SYN Flood saldırı tespit sistemi projesinin genel kapsamını, işlevsel gereksinimlerini ve temel hedeflerini özetlemek amacıyla hazırlanmıştır. Bu raporda sistemin hangi bileşenlerden oluştuğu, hangi verilerle çalıştığı ve kullanıcıya hangi işlevleri sunduğu ana hatlarıyla ortaya konulmaktadır. Proje kapsamında Ubuntu üzerinde kurulu Mininet tabanlı SDN ortamında oluşan ağ trafiği, OS-Ken denetleyicisi aracılığıyla izlenecek; elde edilen akış verileri Python tabanlı backend katmanında işlenerek makine öğrenmesi veya derin öğrenme tabanlı IDS modeli ile analiz edilecektir. Sistem, saldırı tespit sonuçlarını SQLite veritabanında saklayacak ve React tabanlı kullanıcı arayüzü üzerinden kullanıcıya sunacaktır. Proje, hem ağ güvenliği alanında çalışan kullanıcıların hem de teknik düzeyde sistemi izlemek isteyen kişilerin gerçek zamanlı saldırı tespit sonuçlarına erişebilmesini hedeflemektedir.

Proje Kodu	SynGuard-SDN
Proje Adı	Real Time SYN Flood Detection System for Software Defined Networks
Hazırlayanlar	Abdullah Taha Aydın, Eren Eroğlu, Umut Öztürk, Atakan Berber
Başlangıç Tarihi	01.02.2026
Son Değiştirilme Tarihi	27.03.2026
Planlanan Bitiş Tarihi	01.06.2026
Versiyon	1.0
Proje Danışmanı	Dr. Öğr. Üyesi İlker Özçelik
Proje Amacı ve Kapsamı	Ubuntu üzerinde kurulu Mininet tabanlı SDN ortamında, OS-Ken denetleyicisinden elde edilen akış verileri kullanılarak TCP SYN Flood saldırılarını gerçek zamanlı olarak tespit edebilen bir IDS sistemi geliştirilmesi amaçlanmaktadır. Sistem, saldırı sonuçlarını kullanıcı arayüzünde alarm, kayıt ve trafik özeti olarak sunacaktır.
İşlevsel Kapsam	Kullanıcı Doğrulama: Sisteme giriş ve çıkış işlemleri.

(Özet)	
	Veri Toplama: OS-Ken denetleyicisinden akış istatistiklerinin alınması.
	Veri İşleme: Toplanan verilerin ön işleme tabi tutulması ve öznetelik hazırlanması.
	Saldırı Tespiti: ML/DL tabanlı model ile trafiğin normal veya saldırı olarak sınıflandırılması.
	Kayıt Tutma: Saldırı sonuçlarının ve alarm kayıtlarının SQLite veritabanında saklanması.
	Arayüz Sunumu: Sonuçların React tabanlı arayüzde alarm, log ve grafik olarak gösterilmesi.
Kırtasiye ve Sarf İhtiyacı	Projenin tamamı yazılım tabanlı ortamda yürütüleceğinden ek kırtasiye ve sarf ihtiyacı sınırlıdır. Raporlama ve sunum aşamalarında standart çıktı materyalleri gerekebilir.
Donanım Gereksinimleri	Geliştirme/Test: Ubuntu çalıştırabilen bir bilgisayar, Mininet, OS-Ken, Python geliştirme ortamı, React geliştirme araçları ve SQLite için yeterli sistem kaynağı.
	Çalışma Ortamı: Sanal SDN topolojisini ve saldırı senaryolarını çalıştırabilecek yeterli RAM ve işlemci gücü gerekmektedir.
	Genişleme Durumu: Daha büyük trafik senaryoları veya daha ağır model yapıları kullanılması durumunda ek işlem gücü gerekebilir.

B.1.6.3 İş Akış Şemaları

Aşağıda verilen iş akış şeması, geliştirilen SDN tabanlı IDS sisteminin temel çalışma adımlarını göstermektedir. Süreç, kullanıcının sisteme giriş yapmasıyla başlamakta; ardından SDN ortamından akış verilerinin toplanması, bu verilerin işlenmesi, saldırı tespit modeline aktarılması ve elde edilen sonucun kullanıcı arayüzü üzerinden sunulması ile devam etmektedir. İş akış şeması, sistemin hangi sırayla çalıştığını basit ve anlaşılır biçimde ortaya koymakta, böylece hem teknik geliştirme sürecinde hem de analiz aşamasında sistem davranışlarının bütüncül olarak anlaşılmasına yardımcı olmaktadır.



Figür 15. İş Akış Şeması

1. **Kullanıcı Girişi ve Doğrulama:** Kullanıcı, kullanıcı adı ve parola bilgileri ile sisteme giriş yapar. Sistem, girilen bilgileri doğrulayarak yetkili erişim kontrolünü sağlar.
2. **Ana Panelin Açılması:** Giriş işlemi başarılı olduktan sonra kullanıcı ana panele yönlendirilir. Bu panel üzerinden sistem durumu, alarm kayıtları ve trafik özetleri görüntülenebilir.
3. **SDN Ortamında Trafik Oluşumu:** Ubuntu üzerinde kurulu Mininet tabanlı SDN ortamında normal trafik ve saldırı trafiği oluşur. OpenFlow anahtarları bu trafiği iletir.
4. **Akış Verilerinin Toplanması:** OS-Ken denetleyicisi, ağdaki akışlara ait paket sayısı, bayt sayısı, akış süresi, IP ve port bilgileri gibi istatistikleri belirli aralıklarla toplar.
5. **Veri Ön İşleme:** Toplanan ham akış verileri Python tabanlı backend katmanına aktarılır. Bu aşamada veriler temizlenir, düzenlenir ve modelin kullanabileceği formata dönüştürülür.
6. **Özellik Çıkarma:** Ön işlenmiş verilerden saldırı tespitinde kullanılacak öznitelikler hazırlanır. Böylece model için anlamlı giriş verileri oluşturulur.
7. **Saldırı Tespit Modelinin Çalıştırılması:** Hazırlanan öznitelikler, makine öğrenmesi veya derin öğrenme tabanlı IDS modeline gönderilir. Model, ilgili trafiğin normal mi yoksa saldırı davranışı mı içerdiğini belirler.
8. **Sonucun Kaydedilmesi:** Model çıktısı backend tarafından alınır. Saldırı tespiti varsa ilgili kayıtlar SQLite veritabanına alarm ve olay kaydı olarak işlenir.
9. **Sonucun Kullanıcıya Sunulması:** Elde edilen sonuçlar React tabanlı kullanıcı arayüzüne aktarılır. Kullanıcı; alarm bilgilerini, log kayıtlarını ve trafik özetlerini panel üzerinden görüntüler.

B.2.1 İşlevsel Gereksinimler

B.2.1.1. Veri Toplama ve Ön İşleme İşlevleri

- Sistem, OS-Ken denetleyicisi üzerinden akış verilerini belirli zaman aralıklarında toplar.
- Toplanan akış istatistikleri; paket sayısı, bayt sayısı, akış süresi, akış sayısı, kaynak/hedef IP adresleri, port bilgileri ve protokol türü verilerini kapsar.
- Canlı ağ anahtarlarından elde edilmesi zor olan karmaşık istatistikler ve kimlik kolonları veri setinden çıkarılarak ön işleme sürecine tabi tutulur.
- "Saniyedeki Paket Sayısı" ve "Ortalama Paket Boyutu" gibi canlı ağda düşük gecikmeyle hesaplanabilen öznitelikler seçilerek model girdisi oluşturulur.

B.2.1.2. Saldırı Tespit ve Analiz İşlevleri

- Sistem, eğitilmiş derin öğrenme modellerinden en uygununu kullanarak ağ trafiğini "Normal" veya "SYN Flood" olarak gerçek zamanlı sınıflandırır.
- Tespit mekanizması, denetleyici üzerinde ek yük oluşturmayacak ve düşük karar gecikmesi ile çalışacak şekilde optimize edilir.
- Analiz süreci, statik veri setlerinin ötesinde, doğrudan canlı SDN denetleyicisine entegre şekilde anlık trafik akışını değerlendirir.
- Modelin başarısı; doğruluk, kesinlik (precision), duyarlılık (recall), F1-skoru ve gecikme süresi metrikleri üzerinden takip edilir.

B.2.1.3. Kullanıcı Etkileşimi ve Görselleştirme İşlevleri

- Sistem, yetkili kullanıcıların erişimini güvenli hale getirmek amacıyla bir kimlik doğrulama ve giriş mekanizması sunar.
- Dashboard ekranı, ağ üzerindeki trafik yoğunluğunu ve analiz sonuçlarını kullanıcıya anlık ve grafiksel olarak görselleştirir.
- SYN Flood saldırısı tespit edildiği anda, arayüz üzerinde renk değişimi ve görsel uyarılar aracılığıyla gerçek zamanlı alarm üretilir.
- Kullanıcılar, arayüz paneli üzerinden geçmişe dönük saldırı kayıtlarını ve loglanan verileri listeleyebilir.

B.2.1.4. Veri Yönetimi ve Kayıt Tutma İşlevleri

- Tespit edilen tüm saldırı sonuçları ve trafik istatistikleri, SQLite veritabanı yapısı içerisinde düzenli olarak kaydedilir.
- Backend servisi, veritabanı ile kullanıcı arayüzü arasındaki entegrasyonu sağlayarak analiz sonuçlarının izlenebilirliğini garanti altına alır.

B.2.2. Sistem ve Kullanıcı Ara Yüzleri ile İlgili Gereksinimler

B.2.2.1. Kullanıcı Ara Yüzü (UI) Gereksinimleri

- **Kimlik Doğrulama Ekranı:** Sisteme erişim, kullanıcı adı ve şifre bilgilerinin girildiği bir giriş paneli üzerinden sağlanır.
- **Canlı İzleme Paneli (Dashboard):** Ağ trafiği yoğunluğu ve anlık analiz sonuçları, grafiksel bileşenler ve veri tabloları aracılığıyla kullanıcıya sunulur.
- **Alarm Mekanizması:** SYN Flood saldırısı tespit edildiği anda, Dashboard üzerindeki trafik göstergelerinde renk değişimi (yeşilden kırmızıya geçiş) ve uyarı mesajları tetiklenir.
- **Geçmiş Kayıt Görüntüleme:** Kullanıcılar, veritabanında saklanan geçmiş saldırı olaylarını tarih, saat ve saldırı şiddeti detaylarıyla listeleyebilir.

B.2.2.2. Sistem Ara Yüzleri ve Entegrasyon Gereksinimleri

- **Denetleyici Veri Erişimi:** Sistem, trafik verilerini harici bir izleme cihazına ihtiyaç duymadan doğrudan OS-Ken (Ryu) denetleyicisi üzerinden çeker.
- **Backend Entegrasyonu:** Denetleyiciden çekilen akış verileri, Python tabanlı backend servisi aracılığıyla işlenerek analiz birimine aktarılır.
- **Veritabanı Bağlantısı:** Analiz sonuçları ve sistem kayıtları, backend üzerinden SQLite veritabanına loglanır.
- **Model Girdi Arayüzü:** Ön işlemeden geçen ağ istatistikleri, sınıflandırma işlemi için hazırlanan derin öğrenme modeline girdi olarak sunulur.

B.2.3. Veriye İlgili Gereksinimler

B.2.3.1. Veri Kaynağı ve İçerik Gereksinimleri

- Sistem, veri kaynağı olarak doğrudan OS-Ken (Ryu) denetleyicisi üzerinden çekilen akış verilerini kullanır.
- Toplanan veriler; paket sayısı, bayt sayısı, akış süresi, akış sayısı, kaynak ve hedef IP adresleri, port bilgileri ile protokol türü gibi akış istatistiklerini kapsamaktadır.

B.2.3.2. Veri Saklama ve Yapılandırma Gereksinimleri

- Sistem verilerinin kalıcı olarak depolanması ve loglanması amacıyla SQLite veritabanı yapısı kullanılmaktadır.
- Akış istatistikleri, denetleyici tarafından liste yapısında tutulmakta ve analiz için JSON formatında dışarı aktarılmaktadır.
- Analiz sonuçları, trafik verileri ve geçmiş kayıtlar veritabanı üzerinde bütünleşik bir yapıda saklanmaktadır.

B.2.4. Kullanıcılar ve İnsan Faktörü Gereksinimleri, Güvenlik Gereksinimleri

B.2.4.1. Kullanıcı ve İnsan Faktörü Gereksinimleri

- **İzlenebilirlik ve Yönetilebilirlik:** Sistem, teknik analiz yapan bir mekanizma olmanın ötesinde, kullanıcı tarafından kolayca izlenebilir ve yönetilebilir bir yapıya sahiptir.
- **Kullanıcı Odaklı Arayüz:** Tespit edilen saldırı durumları ve ağ trafiği verileri, kullanıcıya anlaşılır ve somut bir biçimde sunulmaktadır.
- **Görsel Geri Bildirim:** Kullanıcı arayüzü; alarm üretimi, trafik izleme ve geçmiş kayıtların görüntülenmesi gibi işlevlerle donatılarak operasyonel izlenebilirlik sağlanmaktadır.
- **Kullanım Amacı:** Geliştirilen prototip; ağ güvenliği araştırmacıları, öğrenciler ve uygulayıcılar için eğitim ve uygulamalı gösterim amaçlı kullanıma uygun tasarlanmıştır.

B.2.4.2. Güvenlik Gereksinimleri

- **Kimlik Doğrulama:** Sisteme yetkisiz erişimi engellemek amacıyla kullanıcı girişi ve kimlik doğrulama mekanizması bulunmaktadır.

B.2.5. Teknik ve Kaynak Gereksinimleri, Fiziksel Gereksinimler

B.2.5.1. Yazılım Gereksinimleri ve Geliştirme Ortamı

- **İşletim Sistemi:** Projenin test ortamı ve simülasyon süreçleri Ubuntu işletim sistemi üzerinde yapılandırılmaktadır.
- **Ağ Simülasyonu:** Gerçekçi bir ağ ortamı oluşturmak amacıyla Mininet yazılımı kullanılmaktadır.

- **SDN Denetleyicisi:** Ağ trafiğinin yönetimi ve veri toplama süreçleri için OS-Ken (Ryu) denetleyicisi tercih edilmiştir.
- **Programlama Dili ve Kütüphaneler:** Sistem bileşenlerinin geliştirilmesinde Python dili kullanılmakta; saldırı tespiti için yüksek işlem hızına sahip derin öğrenme modeli kütüphanesi entegre edilmektedir.
- **Veritabanı Yönetimi:** Analiz sonuçlarının ve kullanıcı verilerinin loglanması için SQLite veritabanı yapısı kullanılmaktadır.
- **Kullanıcı Arayüzü:** İzleme panelinin ve dashboard ekranının geliştirilmesinde React tabanlı kütüphanelerden yararlanılmaktadır.

B.2.5.2. Teknik ve Kaynak Gereksinimleri

- **Veri Seti Kaynağı:** Modelin eğitimi ve test süreçlerinde "SDN-TCP-SYN ATTACK-DDOS DATASET" verisinden faydalanılmaktadır.
- **Ağ İletişimi:** Denetleyici ile backend servisi arasındaki veri aktarımı için Northbound API yapısı kullanılmaktadır.
- **İşlem Kapasitesi:** Canlı akış üzerinde düşük gecikme (latency) ile çalışabilmek adına, donanım yükünü minimize eden ağaç tabanlı algoritmalar tercih edilmektedir.

B.2.5.3. Fiziksel ve Altyapı Gereksinimleri

- **Test Zemini:** Sistem, denetleyici, anahtarlar (switches) ve istemci-sunucu yapılarından oluşan bir ağ simülasyon altyapısına ihtiyaç duymaktadır.
- **Donanım Optimizasyonu:** Derin öğrenme modellerinin aksine, sistemin standart donanım kaynakları üzerinde yüksek performans ve düşük karar gecikmesi ile çalışması hedeflenmektedir.

C. TASARIM

C.1. Sistem Tasarımı

Projenin sistem tasarımı; veri edinimi, analiz ve görselleştirme süreçlerini uçtan uca yöneten katmanlı bir mimari üzerine kurgulanmıştır. Bu yapı, SDN denetleyicisinden gelen ham verilerin işlenerek anlamlı güvenlik raporlarına ve gerçek zamanlı uyarılara dönüştürülmesini sağlar.

C.1.1. Genel Mimari Yapı (Split Architecture)

Sistem, yüksek performans ve düşük gecikme süresi hedefleri doğrultusunda ayrıştırılmış bir mimari (Split Architecture) ile tasarlanmıştır:

- **Sunucu Tarafı (Backend):** Node.js ve Express ortamında geliştirilmiş, WebSocket (Socket.io) tabanlı çift yönlü iletişim katmanıdır. Sistem, SDN donanımlarından ve analiz biriminden gelen anlık ağ akış loglarını ve tehdit verilerini yönlendirerek IDS prensiplerine uygun olarak kalıcı (persistent) bir SQLite veritabanı üzerinde arşivler.
- **İstemci Tarafı (Frontend):** Vite ve React tabanlı geliştirilen modern kullanıcı arayüzü; standart React kancaları (Hook'lar) ile anlık (real-time) durum yönetimini koordine eder. Backend üzerinden WebSocket ile sürekli akan verileri işleyip yüksek duyarlılıktaki dinamik dashboard bileşenlerine kesintisiz bir şekilde aktarır.

- **Analiz Birimi:** Python ortamında geliştirilen derin öğrenme modeli, backend üzerinden gelen akış istatistiklerini sınıflandırarak anlık saldırı olasılık skorlarını ve açıklanabilir yapay zeka (XAI) tanı verilerini üretir.

C.1.2. Fiziksel Veri Akışı ve Katmanlar

Sistemin operasyonel akışı, paylaşılan fiziksel veri akış şemasına uygun olarak dört temel bölgede gerçekleşmektedir:

1. **Ağ Simülasyon Katmanı (Mininet):** Ağ trafiği üretimi bu katmanda başlar. OpenFlow protokolü üzerinden sanal anahtarlar (switch) aracılığıyla akış trafiği (flows) oluşturulur ve yönetilir.
2. **Denetleyici Katmanı (OS-Ken):** Sanal anahtarlardan gelen veriler OpenFlow protokolü aracılığıyla OS-Ken Controller sunucusuna aktarılır. Denetleyici, bu verileri Northbound API üzerinden analiz sunucusuna iletir.
3. **IDS Sunucu Katmanı (Analiz):**
 - a. **Veri Ön İşleme:** Northbound API'den gelen ham veriler, modelin girişine uygun özniteliklere (features) dönüştürülür.
 - b. **Karar Mekanizması:** İşlenen veriler derin öğrenme modeline sunulur; model her akış için bir risk skoru ve XAI tabanlı tanı verisi üretir. Sonuçlar anlık olarak **SQLite** veritabanında loglanır ve eş zamanlı olarak arayüze iletir.
4. **Kullanıcı Etkileşim Katmanı:** Analiz sonuçları ve ağ durumu, React tabanlı dinamik görsel bileşenler üzerinden son kullanıcıya sunulur; kullanıcı sistem üzerinden topolojiyi izler, vaka raporlarını inceler ve acil durum alarmlarını takip eder.

C.1.3. Modüler Bileşen Tasarımı

Sistem tasarımı, kullanıcı deneyimini iyileştirmek ve olaylara müdahale hızını (visibility) en üst düzeye çıkarmak adına modüler bileşenlerden oluşmaktadır:

- **Topology Map Modülü:** SDN donanımı topolojisindeki denetleyici (controller), anahtar (OVS) ve bilgisayar (host) ilişkilerini interaktif olarak sunar. Acil durumlarda ilgili sorunlu düğümü (node) renk değişimleriyle görselleştirir.
- **Live Traffic Monitor Modülü:** WebSocket üzerinden anlık akan ağ trafiği metriklerini, dinamik grafiklerle saniyede bir tazeleyerek aktif paket akışını ve toplam akış (flow) sayısını raporlar.
- **Attack Gauge Modülü:** Derin öğrenme modelinden gelen olasılık verilerini görsel bir gösterge ibresi ile sunar ve yüksek oranlarda (XAI bulgularıyla birlikte) saldırı tehdit seviyesini kullanıcıya duyurarak ekran genelinde "Kırmızı Alarm" (Pulse) efektini tetikler.
- **Incident Archive (Vaka Kaydı) Modülü:** Güvenli veritabanı (SQLite) üzerinden akışları çekerek algılanan tüm tehditleri; zaman damgası, saldırgan IP adresi, atak türü ve acil durum derecesiyle (Critical) listeler.
- **Live Terminal Modülü:** Panelin alt kısmına sabitli log ekranı, OS-Ken denetleyicisindeki düşük seviyeli OpenFlow "Packet-In" etkinliklerini kronolojik olarak sergileyerek ağ mühendislerine teknik izlenebilirlik sağlar.

C.2. Kullanıcı ve Sistem Ara Yüz Tasarımları

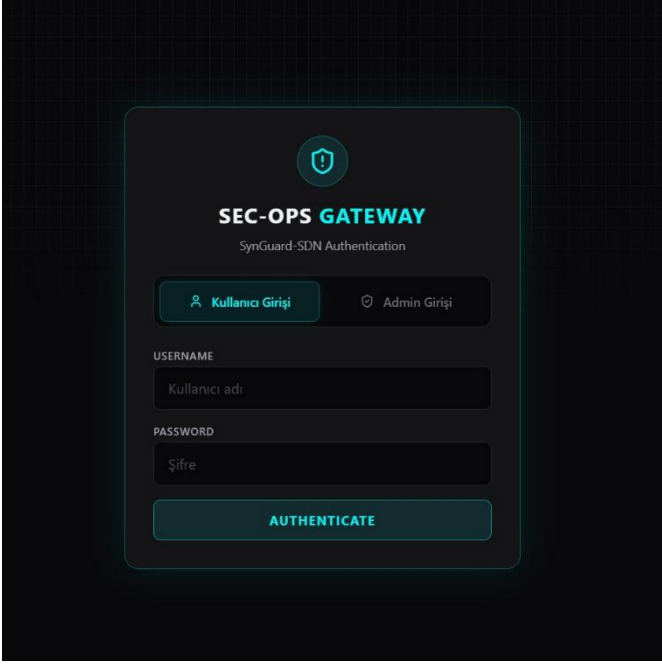
Sistem arayüzü, Software-Defined Networking (SDN) ortamından gelen karmaşık verilerin ve makine öğrenmesi analiz sonuçlarının kullanıcı tarafından hızlıca yorumlanabilmesi amacıyla tasarlanmıştır. Tasarımda yüksek kontrastlı "Neon-Dark" estetiği ve gerçek zamanlı veri görselleştirme bileşenleri ön plana çıkarılmıştır.

C.2.1. Tasarım Prensipleri ve Görsel Dil

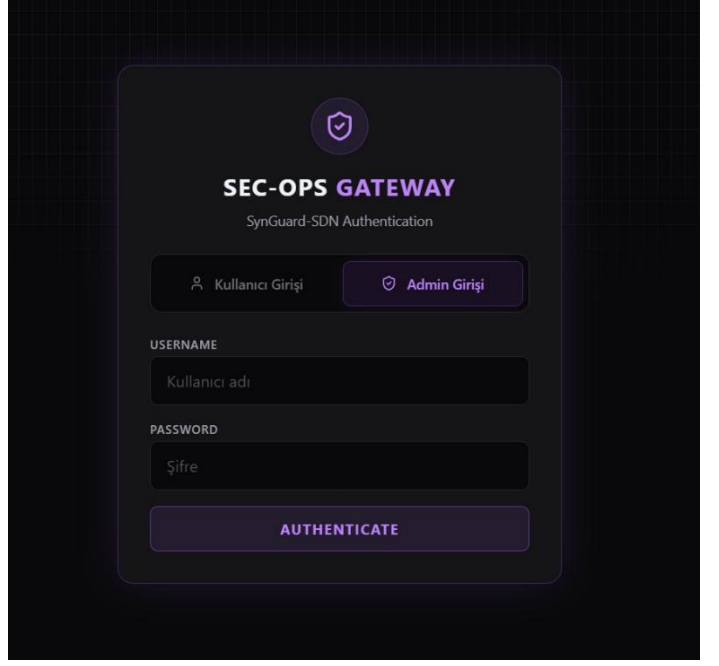
- **Görsel Hiyerarşi:** Arayüz, koyu arka plan üzerinde neon-mavi (cyan) ve kırmızı vurgular kullanılarak geliştirilmiştir. Bu sayede güvenli trafik ile kritik saldırı uyarıları arasındaki görsel ayrım netleştirilmiştir.
- **Veri Akış Yönetimi:** İstemci tarafı, backend servisinden gelen asenkron veri paketlerini ve WebSocket bildirimlerini işleyerek arayüz bileşenlerine anlık olarak yansıtmaktadır.

C.2.2. Ekran Tasarımları ve Bileşen Analizi

C.2.2.1. Kullanıcı Giriş Yapma

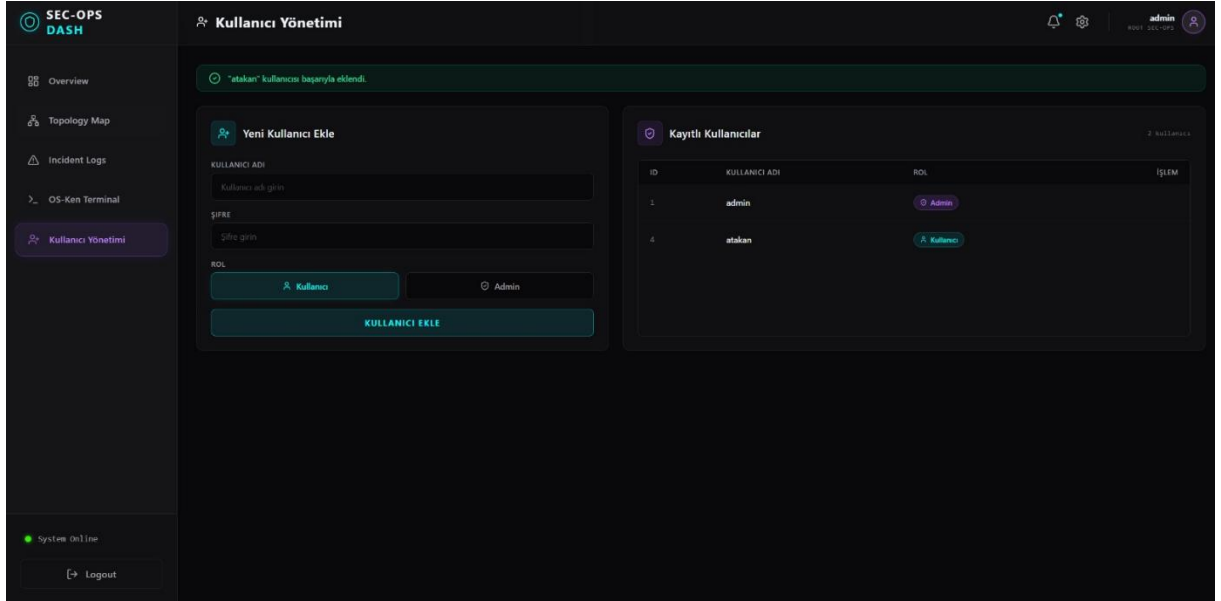


Figür 16. Kullanıcı Giriş Arayüzü



Figür 17. Yönetici Giriş Arayüzü

C.2.2.1. Kullanıcı Kaydı Yapma



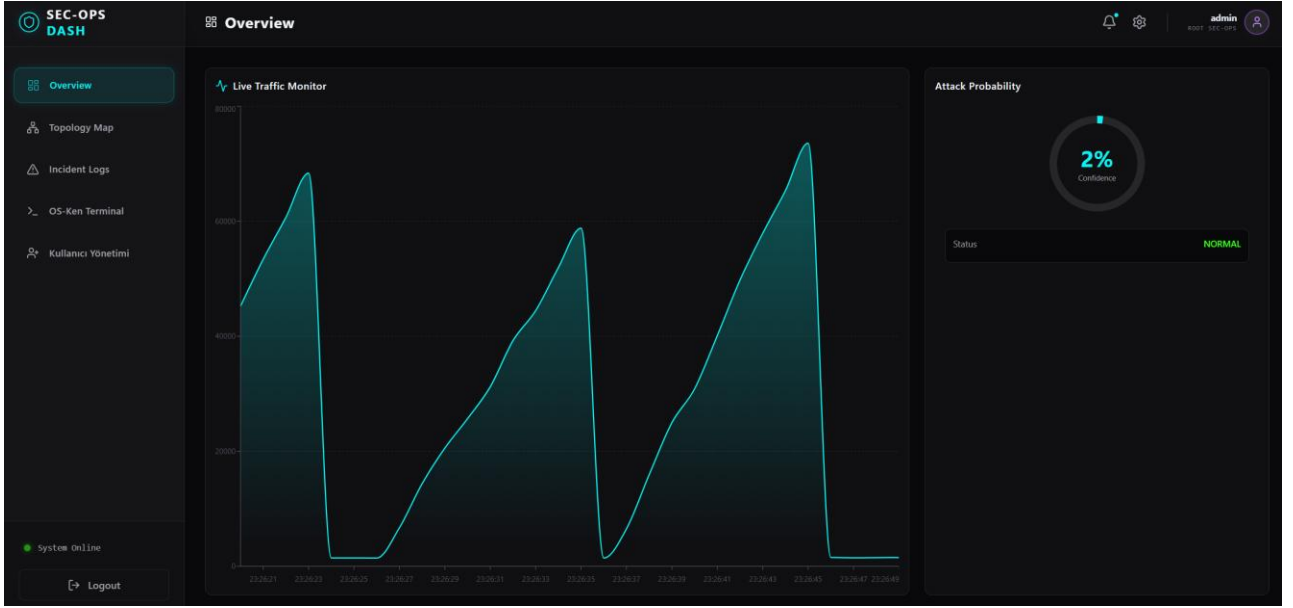
Figür 18. Yönetici Giriş Arayüzü

Bu arayüz sadece yönetici modda görünür olacak şekilde tasarlanmıştır.

C.2.2.2. Genel Bakış ve Canlı İzleme (Overview)

- **Grafik Entegrasyonu:** Recharts kütüphanesi ile oluşturulan "Live Traffic Monitor", ağdaki paket akışını alan grafiği üzerinde saniyelik verilerle sunar.

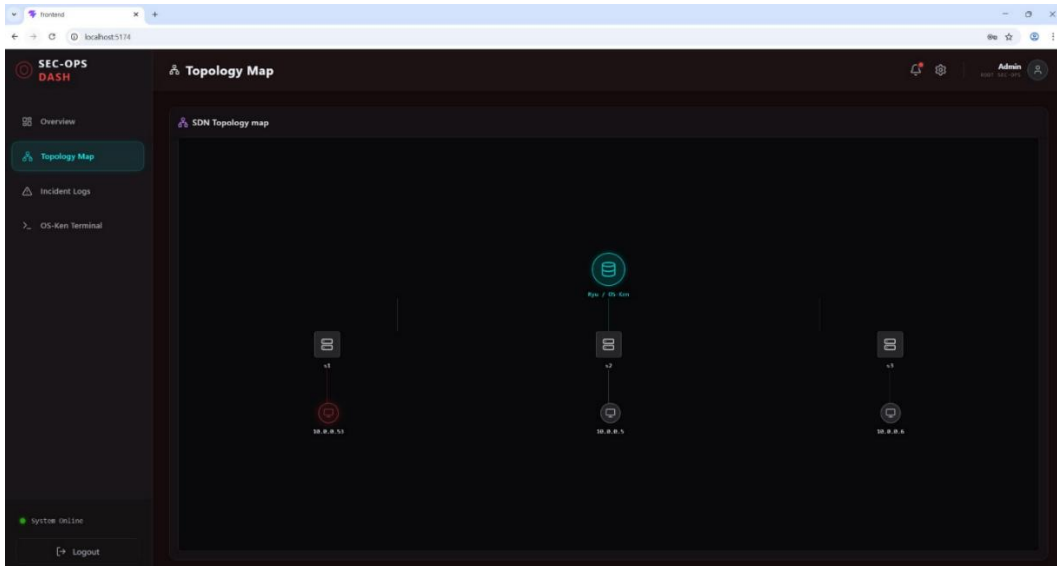
- **Saldırı Olasılık Göstergesi:** "Attack Probability" göstergesi, modelden gelen %0-%100 arası güven skorunu görselleştirir. Düşük olasılıklarda yeşil "NORMAL" statüsü, yüksek olasılıklarda ise kırmızı alarm durumuna geçerek kullanıcıyı uyarır.



Figür 19. Ana Sayfa Arayüzü

C.2.2.3. Ağ Topolojisi ve Tehdit Haritası (Topology Map)

- **Sistem Mimarisi Görselleştirme:** Ryu Denetleyicisi, OpenFlow anahtarları ve uç cihazlar (hosts) arasındaki bağlantılar interaktif bir harita üzerinde sergilenmektedir.
- **Noktasal Tehdit Gösterimi:** Bir saldırı algılandığında, topoloji üzerindeki ilgili uç cihaz (Şekil X'te görülen 10.0.0.53 gibi) kırmızı halka ile işaretlenerek saldırının kaynağı görsel olarak raporlanır



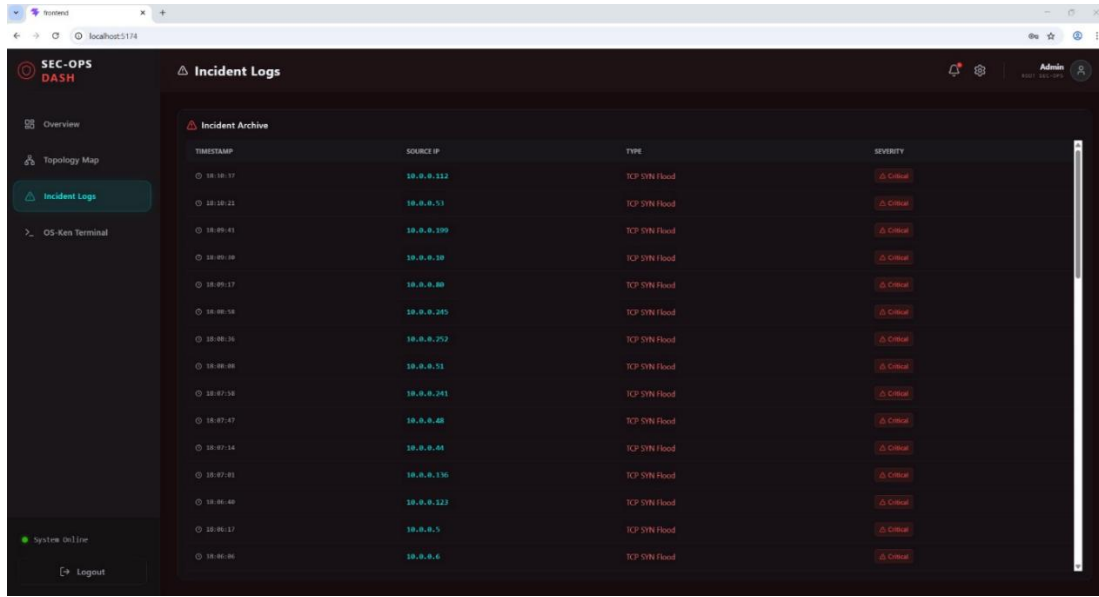
Figür 20. Ağ Topolojisi Arayüzü

C.2.2.4. Olay Arşivi ve Kayıt Yönetimi (Incident Logs)

- **Kronolojik Listeleme:** SQLite veritabanında saklanan geçmiş saldırı kayıtları; zaman damgası,

kaynak IP adresi ve saldırı türü detaylarıyla sunulmaktadır.

- **Vaka Detayları:** Tespit edilen her bir SYN Flood olayı, "Critical" etiketiyle işaretlenerek operasyonel önceliklendirme sağlanmaktadır.

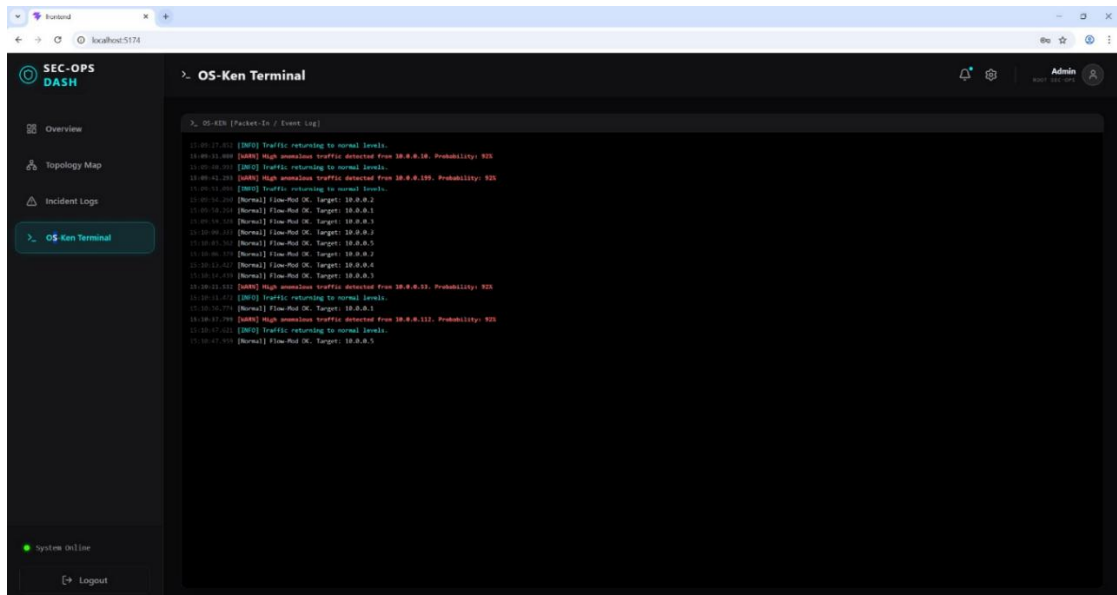


Timestamp	Source IP	Type	Severity
18-10-17	10.0.0.112	TCP SYN Flood	Critical
18-10-21	10.0.0.51	TCP SYN Flood	Critical
18-09-41	10.0.0.109	TCP SYN Flood	Critical
18-09-08	10.0.0.10	TCP SYN Flood	Critical
18-09-17	10.0.0.80	TCP SYN Flood	Critical
18-09-14	10.0.0.245	TCP SYN Flood	Critical
18-08-16	10.0.0.252	TCP SYN Flood	Critical
18-08-08	10.0.0.51	TCP SYN Flood	Critical
18-07-14	10.0.0.241	TCP SYN Flood	Critical
18-07-07	10.0.0.48	TCP SYN Flood	Critical
18-07-14	10.0.0.44	TCP SYN Flood	Critical
18-07-01	10.0.0.135	TCP SYN Flood	Critical
18-06-02	10.0.0.222	TCP SYN Flood	Critical
18-06-17	10.0.0.5	TCP SYN Flood	Critical
18-06-06	10.0.0.6	TCP SYN Flood	Critical

Figür 21. Ağ Topolojisi Arayüzü

C.2.2.5. Teknik İzleme ve Terminal (Live Terminal)

- **Düşük Seviyeli Analiz:** Arayüzün alt kısmında konumlandırılan terminal birimi, denetleyiciden gelen ham "Packet-In" ve "Flow-Mod" olaylarını simüle eder.
- **Anomali Raporlama:** Terminal akışı içerisinde yüksek olasılıklı saldırı tespitleri, XAI tanılarıyla birlikte ("High anomalous traffic detected") kırmızı renkli uyarılarla raporlanmaktadır.



```
OS-KEN [Packet-In / Event Log]

11:01:17.01 [INFO] Traffic returning to normal levels.
11:01:11.400 [WARN] High anomalous traffic detected from 10.0.0.10. Probability: 92%
11:01:10.200 [INFO] Traffic returning to normal levels.
11:01:01.200 [WARN] High anomalous traffic detected from 10.0.0.109. Probability: 92%
11:01:11.400 [INFO] Traffic returning to normal levels.
11:01:10.200 [Normal] Flow-Ret OK. Target: 10.0.0.0
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.1
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.2
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.3
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.4
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.5
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.6
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.7
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.8
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.9
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.10
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.11
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.12
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.13
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.14
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.15
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.16
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.17
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.18
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.19
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.20
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.21
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.22
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.23
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.24
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.25
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.26
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.27
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.28
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.29
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.30
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.31
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.32
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.33
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.34
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.35
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.36
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.37
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.38
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.39
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.40
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.41
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.42
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.43
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.44
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.45
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.46
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.47
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.48
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.49
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.50
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.51
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.52
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.53
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.54
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.55
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.56
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.57
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.58
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.59
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.60
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.61
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.62
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.63
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.64
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.65
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.66
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.67
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.68
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.69
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.70
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.71
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.72
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.73
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.74
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.75
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.76
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.77
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.78
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.79
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.80
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.81
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.82
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.83
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.84
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.85
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.86
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.87
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.88
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.89
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.90
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.91
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.92
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.93
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.94
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.95
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.96
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.97
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.98
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.99
11:01:10.100 [Normal] Flow-Ret OK. Target: 10.0.0.100
```

Figür 22. Ağ Topolojisi Arayüzü

C.3. Test Tasarımı

Sistemin kararlılığını, veri bütünlüğünü ve saldırı tespit başarısını doğrulamak amacıyla katmanlı bir test stratejisi izlenmiştir. Test süreci; gereksinimlerin karşılanmasından uçtan uca sistem performansına kadar

geniş bir yelpazeyi kapsamaktadır.

C.3.1. Gereksinim Analizlerinden Teste Yönelik Hedeflerin Detaylandırılması

Test süreci, **B.2. Analiz** bölümünde belirlenen gereksinimlerin doğrulanmasını hedefler. Bu kapsamda şu spesifik hedefler belirlenmiştir:

- **Kimlik Doğrulama Hedefi:** Yetkisiz erişimlerin engellendiğinin ve **JWT (JSON Web Token)** akışının tarayıcı tarafında güvenli yönetildiğinin doğrulanması.
- **Tespit Doğruluğu Hedefi:** Derin öğrenme modelinden gelen olasılık skorlarının, eşik değerleri aştığında (örn: %80+) arayüzde doğru alarm seviyesini tetiklediğinin kanıtlanması.
- **Veri Kalıcılığı Hedefi:** Tespit edilen her saldırının, kayıp yaşanmadan **SQLite** veritabanına tüm parametreleriyle (Zaman, IP, Tür) işlendiğinin teyidi.
- **Görsel Bildirim Hedefi:** Saldırı anında topoloji haritası ve dashboard üzerindeki görsel efektlerin (kırmızı pulse) senkronize çalıştığının izlenmesi.

C.3.2. Fonksiyonel Test Tasarımı

Sistemin temel işlevleri, hazırlanan senaryo bazlı testlerle kontrol edilir:

- **WebSocket El Sıkışma Testi:** İstemci ve sunucu arasındaki **Socket.io** bağlantısının kararlılığı; sayfa yenileme veya ağ kopmaları sonrası otomatik yeniden bağlanma (auto-reconnect) özellikleri üzerinden test edilir.
- **Canlı Veri Akış Testi (1Hz Flow):** Backend tarafından üretilen simüle akış verilerinin, **Recharts** grafiklerinde saniyede bir kez (1Hz) gecikmesiz olarak render edildiği doğrulanır.
- **Terminal Olay Kaydı Testi:** OS-Ken denetleyicisinden gelen ham "Packet-In" loglarının, terminal bileşeninde kronolojik ve doğru renk kodlarıyla (Mavi: Bilgi, Kırmızı: Saldırı) gösterilmesi test edilir.
- **Vaka Arşivleme Testi:** "Incident Logs" sayfasında yapılan sorgulamaların, veritabanındaki gerçek verileri doğru filtrelerle getirip getirmediği manuel olarak kontrol edilir.

C.3.3. Performans Test Tasarımı

Sistemin gerçek zamanlılık iddiası şu performans metrikleri üzerinden test edilir:

- **Uçtan Uca Gecikme (End-to-End Latency):** Bir saldırının simüle edildiği an ile Dashboard üzerinde "Kırmızı Alarm" efektinin tetiklendiği an arasındaki süre ölçülür. Hedef, bu sürenin **500ms** altında kalmasıdır.
- **Veri İşleme Kapasitesi:** Saniyede gelen akış veri paketi sayısı artırılarak (stres testi), React arayüzünün donma yaşamadan grafikleri güncelleyebilme kapasitesi gözlemlenir.
- **Veritabanı Yazma Performansı:** Yoğun saldırı simülasyonları altında (saniyede birden fazla atak kaydı), **SQLite** dosyasının kilitlenmeden (locking) verileri sırayla işleme yeteneği doğrulanır.

C.4. Yazılım Tasarımı

Sistemin yazılım tasarımı; modülerlik, sürdürülebilirlik ve sorumlulukların ayrılması (Separation of Concerns) prensipleri doğrultusunda yapılandırılmıştır. React tabanlı frontend ve Node.js tabanlı backend

birimleri, birbirlerinden bağımsız geliştirilebilir ve ölçeklenebilir bir mimari sunar.

C.4.1. Gereksinime Bağlı Tasarım Kalıpları Seçimi

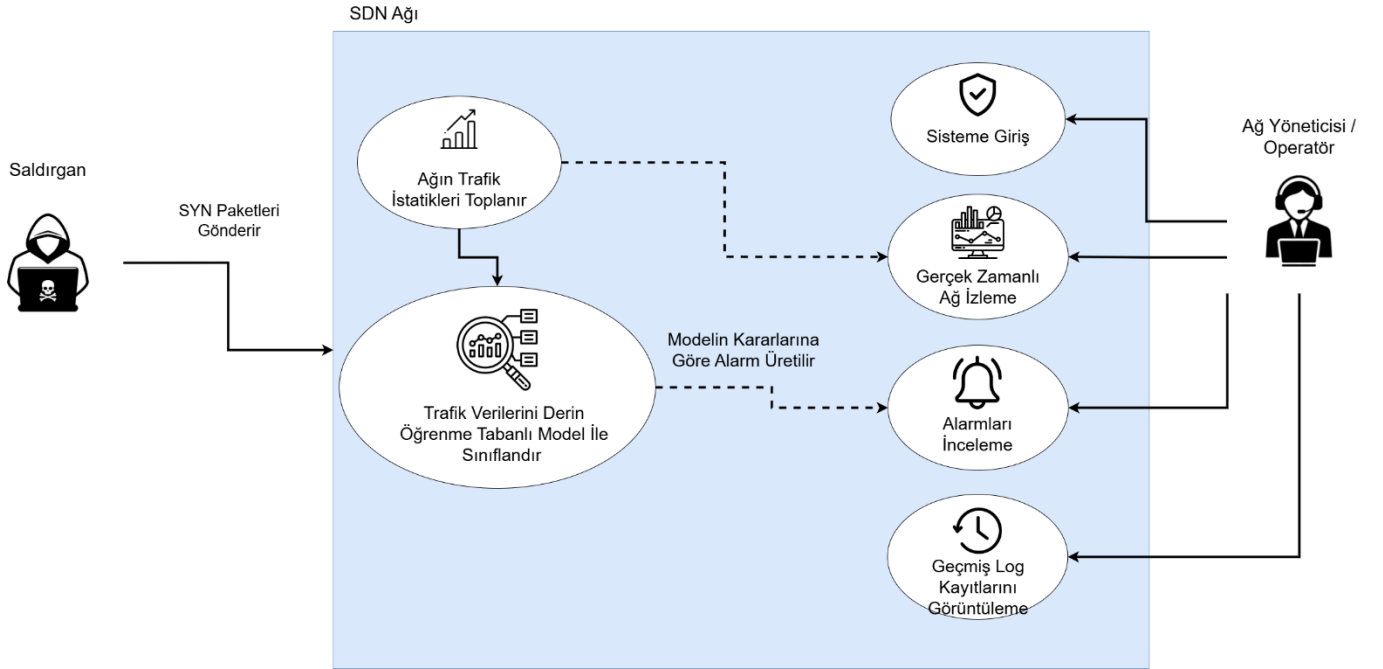
Projenin gerçek zamanlı izleme, ağ topolojisi yönetimi ve güvenlik analizi gereksinimlerini karşılamak amacıyla aşağıdaki tasarım kalıpları (Design Patterns) tercih edilmiştir:

- **İstemci-Sunucu (Client-Server) Mimarisi:** Frontend ve Backend birimlerinin ayrılması, arayüzün (SEC-OPS DASH) bağımsız olarak geliştirilmesini sağlar. Bu yapı, ileride sistemin farklı ağ denetleyicileriyle (Ryu harici) entegrasyonunu kolaylaştırır.
- **Observer (Gözlemci) Kalıbı:** Socket.io üzerinden uygulanan bu kalıp, sunucu tarafında meydana gelen her bir ağ olayının (saldırı tespiti, trafik dalgalanması) anında tüm bağlı arayüz bileşenlerine (Grafik, Terminal, Harita vb.) eş zamanlı olarak duyurulmasını sağlar.
- **Component-Based (Bileşen Tabanlı) Tasarım:** React üzerinde uygulanan bu yapı ile her bir dashboard ögesi (Attack Gauge, Topology Map, Incident Table vb.) kendi içinde kapsüllenmiş (encapsulated) ve bağımsız test edilebilir modüller olarak tasarlanmıştır.
- **Singleton (Tekil) Pattern:** Veritabanı bağlantı yönetimi (db.js) ve WebSocket sunucusu başlatma süreçlerinde, sistem kaynaklarının verimli kullanılması adına tüm uygulamanın tek bir örnek üzerinden işlem yapması sağlanmıştır.

C.4.2. UML Kullanarak Tasarım Diyagramları Oluşturma

Sistemin yazılım bileşenleri, kullanıcı etkileşim senaryoları ve asenkron veri akışları arasındaki ilişkileri standartlaştırmak amacıyla endüstri standartlarında çeşitli UML (Birleşik Modelleme Dili) diyagramları tasarlanmıştır:

C.4.2.1. Kullanım Senaryosu (Use Case) Diyagramı



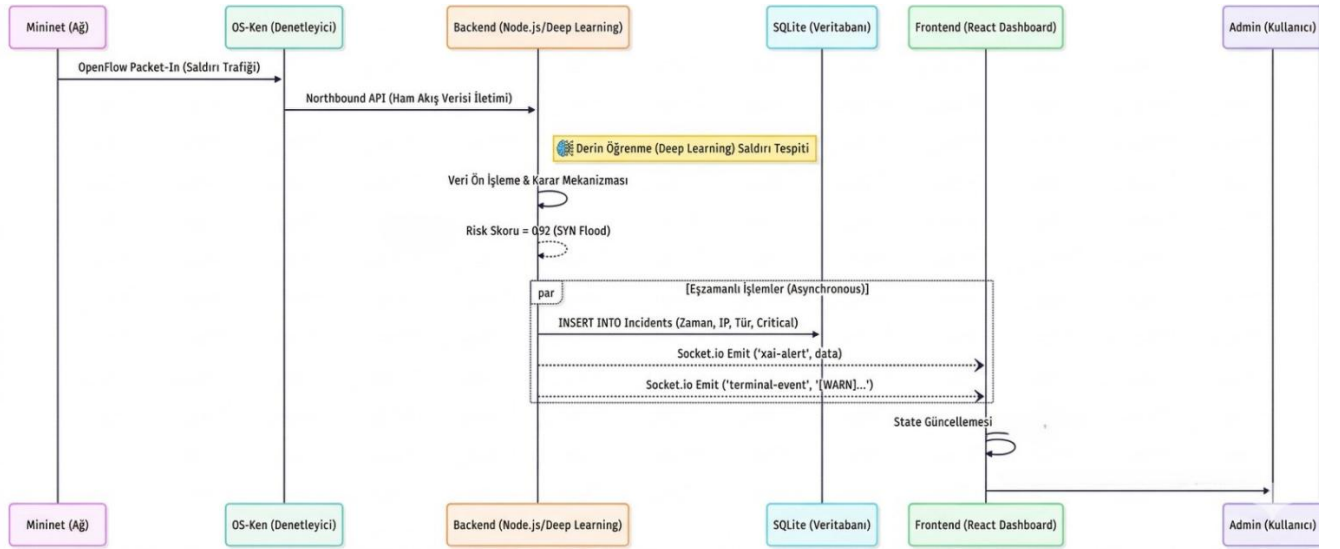
Figür 23. Use-Case Diyagramı

Geliştirilen SDN tabanlı otonom saldırı tespit sisteminin dış faktörlerle (aktörlerle) etkileşimi ve kendi içindeki modüler karar mekanizmaları **Kullanım Durumu (Use Case) Diyagramı** ile modellenmiştir.

Diyagram, sistemin sınırlarını netleştirerek aktörlerin rolünü ve fonksiyonlar arası bağımlılıkları şu şekilde ortaya koymaktadır:

- **Aktörler:** Sistemde iki ana dış aktör bulunmaktadır. Ağ Yöneticisi (Admin), web arayüzü (React Dashboard) üzerinden ağ trafiğini canlı olarak izleyen ve üretilen logları görüntüleyen yasal kullanıcıdır. Saldırgan (Hacker - Hping3) ise dış ağdan (Mininet ortamından) sisteme paket göndererek trafik yoğunluğunu tetikleyen ve sistemin tespit mekanizmalarını harekete geçiren test aktörüdür.
- **Zorunlu Bağımlılık (<<include>> İlişkisi):** Diyagramdaki en temel mühendislik kurallarından biri, *Trafiği Sınıflandırma* işleminin *Trafik İstatistiklerini Toplama* işlemine <<include>> okuyla bağlanmasıdır. Bu durum, derin öğrenme modelinin tahminde bulunabilmesi için SDN denetleyicisinden (OS-Ken) o 5 temel özneteliğin (saniyedeki paket, ortalama boyut vb.) çekilmesinin mecburi bir ön koşul olduğunu göstermektedir.
- **Şarta Bağlı Uzantılar (<<extend>> İlişkisi):** Sistemin otonom karar mekanizması <<extend>> ilişkileriyle tasarlanmıştır. Derin öğrenme modeli her bir akışı sınıflandırır; ancak *Zararlı IP'yi Engelleme (DROP)* ve *Alarmları Görüntüleme* işlemleri her zaman çalışmaz. Bu iki işlem, yalnızca modelin ağ trafiğini "Sınıf 1 (Saldırı)" olarak etiketlemesi koşuluyla (Saldırı Tespit Edilirse) ana sınıflandırma sürecine dahil olan opsiyonel uzantılardır.

C.4.2.2. Bileşen (Component) Diyagramı



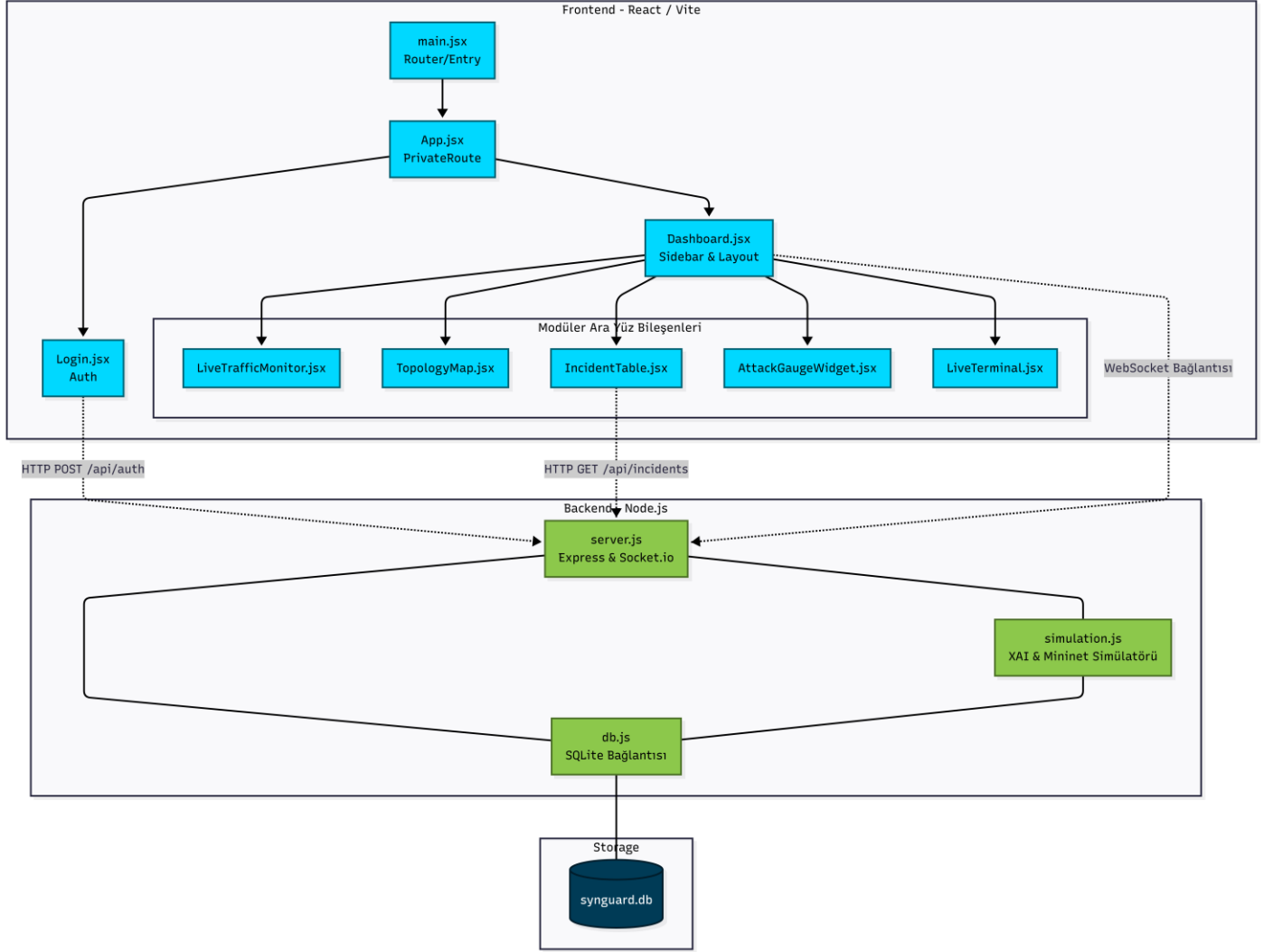
Figür 24. Bileşen Diyagramı

Sistemin İstemci (Frontend), Sunucu (Backend) ve Veritabanı katmanlarındaki modüller yapısının birbirleriyle olan fiziksel ve mantıksal haberleşmesini temsil eder.

- **Frontend Hiyerarşisi:** main.jsx ana yönlendirici (Router) görevini üstlenirken; arayüz Dashboard.jsx içerisine gömülü navigasyon mekanizması üzerinden LiveTrafficMonitor, TopologyMap, IncidentTable, AttackGaugeWidget ve LiveTerminal gibi bağımsız ve kapsüllenmiş (encapsulated) görsel modüllere ayrılmıştır .
- **Backend Etkileşimi:** İstemci tarafındaki "Login" modülü geleneksel HTTP POST (/api/auth) ile, Log tablosu ise HTTP GET (/api/incidents) metodolojisi ile haberleşirken; anlık canlı dashboard

render işlemleri doğrudan server.js üzerinden açılan kesintisiz WebSocket köprüsüyle beslenmektedir .

C.4.2.3. Sıralama ve Akış (Sequence) Diyagramı



Figür 25. Sıralama ve Akış (Sequence) Diyagramı

Sistemin "Gerçek Zamanlı (Real-Time)" otonom olay tepki süresini (Event-Driven Architecture) ardışık zaman çizelgesinde kanıtlamak üzere tasarlanmıştır. Bu şema, tespit edilen bir anomali durumundaki "Uçtan Uca" paket yolculuğunu gösterir:

- **Veri Girişi:** Mininet simülasyonundan doğan sentetik ağ trafiği (Örn: TCP SYN Flood), denetleyici (OS-Ken) üzerinden Northbound API ile Node.js sunucusuna aktarılır.
- **Karar Mekanizması:** Sunucu katmanında verilerin ön işleme yapılarak derin öğrenme modelinin algoritmasından geçirilir ve yüksek bir risk skoru üretilir.
- **Eşzamanlılık (Paralel İşlem / par Bloğu):** Bu aşamadan sonra sistem senkron çalışmayı bırakarak asenkron (paralel) yayına geçer. Arka planda IDS statüsü gereği saldırı anında SQLite veritabanının Incidents tablosuna saldırgan IP ve türüyle kalıcı kayıt (INSERT) atılırken; aynı milisaniyeler içerisinde kullanıcıyı bekletmeksizin arayüze (Frontend) Socket.io üzerinden xai-alert ve [WARN] terminal logları fırlatılır (Emit).
- **Son Evre:** İstemci arayüzü, aldığı bu WebSocket uyarısıyla kendi durumunu (state) günceller (isAlerting = true) ve ekran genelinde kırmızı alarm (Pulse Effect) ile Gauge (Kadran) güncellemesi

tetiklenerek süreç tamamlanır.

C.5. Veri Tabanı Tasarımı

Sistemin veri kalıcılığı (persistence) katmanı, düşük gecikme süresi ve taşınabilirlik avantajları nedeniyle ilişkisel bir veritabanı olan SQLite üzerinde yapılandırılmıştır. Tasarım; ağ akış verilerinin bellek yormadan yüksek hızda kaydedilmesi ve geçmiş saldırı olaylarının (incidents) arayüz üzerinden hızlıca sorgulanması hedefleri doğrultusunda optimize edilmiştir.

C.5.1. Veri Tabanı İsterleri ve Tablo Dökümü

Sistemde olayları ve istatistikleri loglamak için kullanılan 3 temel tablo, içerdikleri alanlar ve veri tipleri aşağıda detaylandırılmıştır:

C.5.1.1. Users Tablosu (Kullanıcı Yönetimi)

Sisteme yetkisiz erişimleri engellemek için kimlik bilgilerini barındıran tablodur.

Alan Adı	Veri Tipi	Kısıt (Constraint)	Açıklama
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Benzersiz kullanıcı ID numarası
username	TEXT	UNIQUE, NOT NULL	Sisteme özel kullanıcı adı
password_hash	TEXT	NOT NULL	Güvenlik için hashlenerek şifrelenmiş parola

C.5.1.2. TrafficLogs Tablosu (Ağ Akış Arşivi)

Grafik modüllerinin kronolojik veriye ulaşabilmesi için OS-Ken üzerinden gelen saniyelik ağ trafik sayımlarını biriktiren depodur.

Alan Adı	Veri Tipi	Kısıt (Constraint)	Açıklama
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	İlgili zaman diliminin log numarası
timestamp	TEXT	NOT NULL	Metin formatında (ISOString) saniyelik kayıt zamanı
packet_flow	INTEGER	NOT NULL	Saniyedeki anlık paket akış adedi
active_flows	INTEGER	NOT NULL	Denetleyici üzerindeki aktif bağlı donanım/akış sayısı

C.5.1.3. Incidents Tablosu (Saldırı Kayıtları)

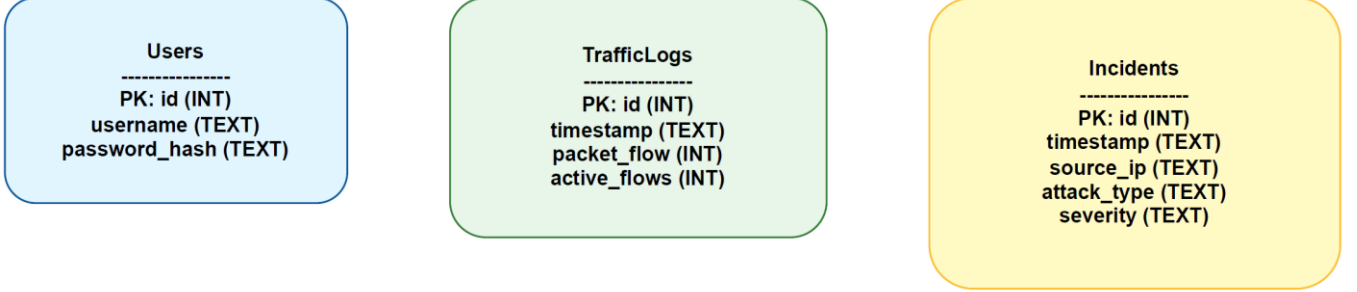
XAI (Açıklanabilir Yapay Zeka) destekli model tarafından anomali olarak etiketlenen tüm durumların arşivlendiği vaka tablosudur.

Alan Adı	Veri Tipi	Kısıt (Constraint)	Açıklama
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Benzersiz vaka kayıt numarası
timestamp	TEXT	NOT NULL	Saldırının tespit edildiği an (ISOString formunda)
source_ip	TEXT	NOT NULL	Atak üreten saldırganın/hedefin tespit edilen IP adresi
attack_type	TEXT	NOT NULL	Atak türü (Örn: TCP SYN Flood)
severity	TEXT	NOT NULL	Tehdit derecesi (Critical, High vb.)

C.5.2. E/R (Varlık-İlişki) Yapısı

Sistemin E/R diyagram yapısı, IDS (Saldırı Tespit Sistemi) doğasına uygun olarak varlıklar arasındaki mantıksal ve kronolojik bağları sergiler:

- **Users** tablosu tamamen bağımsız olup yalnızca sistem oturumunu kontrol eder.
- **TrafficLogs** (Trafik Logları) ve **Incidents** (Saldırı Vakaları) tabloları arasında **zayıf bir 1:N (Bire Çok)** varlık ilişkisi kurgulanmıştır.
- **İlişki Mantığı:** Veritabanında saniyede bir oluşan her trafik kaydı mutlaka bir saldırı (Incident) değildir. Ancak tespit edilen her saldırı kaydı, zorunlu olarak o andaki yüksek yoğunluklu trafik verisinden (TrafficLogs varlığından) köken almaktadır.



Figür 26. Veri Tabanı Varlık Diyagramı

C.5.3. İş Mantığı, Kısıt (Constraint) ve Tetikleyici (Trigger) Tasarımları

SQLite mimarisi, büyük veritabanı motorlarındaki (PostgreSQL/Oracle) **Stored Procedures (Saklı Yordamlar)** yapısına natürel olarak sahip değildir. Bu nedenle projenin temel iş mantığı (Business Logic) doğrudan **Node.js/Express uygulama katmanına** delege edilmiştir:

- **Audit & Log Entry Mekanizması:** Incidents tablosuna bir saldırı (Critical severity) kaydı işlendiği anda, WebSocket yayını (socket.emit) ile uygulama katmanı tetiklenir ve bu bilgi anında canlı arayüzde bir "Alarm Log Entry" olarak belirir.
- **Kısıt (Constraint) Yönetimi:** Tüm tablolardaki elzem verilere (timestamp, source_ip, severity vb.) schema düzeyinde **NOT NULL** koruması atanarak eksik veri paketleriyle (Null Data) gelen sorguların kayıt bütünlüğünü (Data Integrity) bozması engellenmiştir.
- **Veri Yüğü Optimizasyonu (Cleanup Storage):** Demo ve gerçek zamanlı analiz ortamında diskin şişmesini önlemek amacıyla, TrafficLogs tablosuna yapılan arka arkaya ekleme (Insert) sorguları Node.js uygulaması (backend loop) aracılığıyla sadece en güncel durumu kapsayacak şekilde sınırlandırılmaktadır.

D. REFERANSLAR

- Arvind, T., & Radhika, K. (2023). *XGBoost machine learning model-based DDoS attack detection and mitigation in an SDN environment*.

- Ji, W., Yang, Y., Zhang, Y., Wang, Y., Tian, M., & Qiu, Y. (2023). *DDoS attack detection based on information entropy feature extraction in software defined networks*.
- Das, T., Abu Hamdan, O., Sengupta, S., & Arslan, E. (2022). *Flood control: TCP-SYN flood detection for software-defined networks using OpenFlow port statistics*.
- Espinoza Godínez, A., Garza Rivera, J. L., & Tinoco Varela, D. (2021). *Detection of DoS and DDoS attacks using LSTM in computer networks: A systematic literature review*.
- Hirsi, A., Alhartomi, M. A., Audah, L., Hamdi, M. M., Salah, A., Ansa, G. O., Ahmed, S., & Farah, A. (2022). *Hybrid CNN-LSTM model for DDoS detection and mitigation in software-defined networks*.
- Ahuja, N., Mukhopadhyay, D., & Singal, G. (y1l). *DDoS attack traffic classification in SDN using deep learning*.